

Verification Continuum™

Verdi® nAnalyzer

User Guide and Tutorial

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Audience	6
Customer Support	6
Synopsys Statement on Inclusivity and Diversity	7

1. Introduction	8
Overview - Why nAnalyzer?	8
Technology	8
Timing Analysis Debug	8
Clock/Reset Tree Analysis	9
Switching Analysis	9
Before You Begin	9

2. Timing Analysis Debug	11
Overview	11
Standard Delay Format (SDF)	11
Timing Analysis Results	12
Use Models	12
Standard Delay Format (SDF)	12
Timing Analysis Results	15
Tutorials	16
Use SDF Data to Determine Longest Path	17
Load the SDF File	17
Trace the Logic between Two Points	18
Calculate Longest Path	21
Debug PrimeTime Results	25
Load the PrimeTime Results	25
Show the Results on nSchema	27
Isolate the Logic	29
Compare with SDF	30
Attach Clock Domain Information	31

3. Clock/Reset Tree Analysis	33
---	-----------

Contents

Overview	33
Clock/Reset Tree	33
Clock Domain Analysis	34
Clock Domain Analysis Background	34
Terminology	34
Clock Signal	34
Internal Clock Source	35
Clock Domain	35
Clock Source Tree	35
Clock Source	38
Extract Clock Source Tree	39
Constant Value Propagation	39
Extract Clock Domain	40
Pass Through Logic	40
Build Clock Source Tree	40
Specify Clock Source	41
Mix-Clock Domain Crossing-Path	41
Tutorials	43
Display a Clock Tree	43
Analyze Clock Skew	47
Display a Reset Tree	48
Understand and Debug Clock Domains	49
Extract Clock Domains	49
View Clock Domain Results in <i>nSchema</i>	51
Further Constrain the Clock Domains	52
Highlight Clock Domains	55
Find the Clock Source	56
Analyze Clock Domains for Crossing Paths	58
Check for Crossing Paths	58
View Crossing Paths	59
Create a User-Defined Synchronizer	62
4. Switching Analysis	66
Overview	66
Tutorial	66
Analyze Switching Activity on RTL Design	67
5. Appendix A: Synchronizer Template	72

Contents

Overview	72
Elements in Synchronizer Template	72
StartPoint	72
EndPoint	73
CrossPath	73
FromDomain	73
ToDomain	73
Template	73
Template Types	73
Synchronizer Module Template	74
Flatten Synchronizer Template	74
Default Template	75
Template sync_1 Type A & B	75
Template sync_2 Type A & B	76
Mux (D-Mux) Synchronizer	77
Template sync_module	78

6. Appendix B: CDC Correlation	79
Convergence	79
Divergence	80
Reconvergence	81

Preface

This book is designed to allow you to quickly become proficient in the *nAnalyzer* module, an add-on to the Verdi platform. You should already be familiar with the Verdi platform before beginning this book. If you do not feel comfortable with basic Verdi operations, please review the *Verdi User Guide and Tutorial* document first.

This book focuses on the most commonly used commands without going into detail on everything. For detailed descriptions of individual commands, please refer to the appropriate chapter of the *Verdi and Siloti Command Reference Manual*.

The manual should be read from beginning to end, although you may skip any sections with which you are already familiar.

Audience

The audience for this manual includes engineers who are familiar with languages and tools used in design and verification and are frequently required to do further analysis of clock and reset trees, timing issues and signal switching activity.

This document assumes that you have a basic knowledge of the platform on which your version of the Verdi platform runs: UNIX or Linux and that you are knowledgeable in Verilog or VHDL, simulation software, and digital logic design.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product - L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

This section describes the following topics:

- [Overview - Why nAnalyzer?](#)
- [Technology](#)
- [Before You Begin](#)

Overview - Why nAnalyzer?

The Verdi platform enables engineers to locate, isolate, understand, and resolve functional (simulation) bugs in a fraction of the time of traditional solutions; however, in addition to functional debug, there is also a portion of the process generally dedicated to inspecting, analyzing, and debugging the implementation aspects of a design, as opposed to the functional aspects of a design. Although there is not always a clean separation between these two domains, implementation includes many other analysis tasks that need to be “signed off” before a design is considered complete. These analysis tasks include analyzing clock/reset trees and clock domains, debugging timing analysis failures and understanding signal switching activity. Support for these tasks have been combined in the *nAnalyzer* module.

The analysis functions have been layered on top of existing Verdi components as other menu items. These menu items may open other related windows. The combination of the Verdi platform and the *nAnalyzer* module maximizes the efficiency and productivity of expensive engineering resources, significantly reduces costs, and dramatically accelerates the process of getting silicon to market.

Technology

The *nAnalyzer* module provides support in the following implementation related areas.

Timing Analysis Debug

As die size continues to decrease and speed continues to increase, the effects of parasitics on timing becomes critical. The ability to view and understand the timing is

important. The *nAnalyzer* module provides the following functionality in the area of timing analysis debug:

- Load Standard Delay Format (SDF) files and annotate the delay information onto *nSchema*.
- Load timing results from standard timing tools (e.g. Synopsys' PrimeTime), view a list of critical paths, and display a flat schematic view for the critical paths.
- Perform simple delay analysis (e.g. longest/shortest path between points).

Clock/Reset Tree Analysis

During the design process, you'll want to investigate the clock and reset trees to confirm proper balancing. The *nAnalyzer* module provides the following functionality in the area of clock/reset tree analysis:

- Create a schematic view displaying the clock/reset tree and then annotate timing for skew analysis.
- Analyze the different clock domains in your design.
- Check clock domains for proper synchronization and locate crossing paths.

Switching Analysis

As more applications require low power consumption and design schedules are compressed, waiting until the end of the design cycle for full power analysis is not always practical. With switching analysis, you can investigate signal activity early in the design cycle. Using the design data and FSDB file, the *nAnalyzer* module provides the following functionality:

- Determine which nets have the most transitions over a specified time range (switching activity).
- Identify the times with the most transitions (peak activity).

Before You Begin

Before you begin any of the tutorials, you (or your system manager) must have installed the Verdi platform (which automatically installs *nAnalyzer*) as described in the accompanying *Installation and System Administration Guide*. You must also complete the

following actions in order to set up the Verdi environment and the files required for this tutorial:

1. Add the Verdi application (binary) to the search path and specify the search path to the license file:

Refer to the *Setting Up the Environment and Running the Software* section in the *Installation and System Administration Guide* for details.

Note:

Your license file must include an *nAnalyzer* feature line for this tutorial.

2. Set up the symbol library environment variables:

Typically you must build a gate-level symbol library for your design using the following command:

```
syn2SymDB synopsys.lib
```

Note:

The `synopsys.lib` is not included in the Verdi package. It is available from a Synopsys tool package.

For these examples, the symbol library is already built and is included in the installation.

```
% setenv NOVAS_LIBPATHS $VERDI_HOME/share/symlib/xx
% setenv NOVAS_LIBS lsi10k_u
```

where `xx` will be either 32 or 64 depending on your system.

3. Create a working directory:

```
% mkdir <working_dir>
```

4. All of the tutorial data resides in the `$VERDI_HOME/demo` directory. Make a copy of these demo files in your working directory:

```
% cp -r $VERDI_HOME/demo <working_dir>
```

2

Timing Analysis Debug

This section describes the following topics:

- [Overview](#)
- [Use Models](#)
- [Tutorials](#)

Overview

In the area of timing analysis debug, *nAnalyzer* supports two primary functions: viewing and analyzing Standard Delay Format (SDF) data in the context of the design and debugging timing analysis results.

Standard Delay Format (SDF)

Depending on a company's design flow one sign-off criteria can be a successful gate-level timing simulation with SDF enabled. The chip's timing must be accurate to meet setup and hold times of registers and the timing conditions of memories. Trying to visualize and understand the SDF is tedious and time consuming.

The two main requirements for analyzing SDF data are a gate-level design with a corresponding SDF file. You may also have an FSDB file that was generated with SDF enabled. After loading this information in the Verdi platform, you have several options for using the *nAnalyzer* module to view and analyze SDF:

- View the delay paths (PATHCONSTRAINT in the SDF) in the *nSchema* frame.
- Annotate the interconnect and cell delay (INTERCONNECT and IOPATH in the SDF) on *nSchema* with the following options:
 - Select the preferred corner case (if multiple exist in the SDF).
 - Modify the precision.
 - Check the cell delay.

- Calculate the longest/shortest paths between two points and display the results textually or schematically.
- View the SDF annotated FSDB results in *nWave*.

Note:

The Verdi platform currently supports version 3.0 as the primary supported version. Older versions are also supported to allow existing SDF files to be used without any modification.

A warning message will be displayed to let you know which version is parsed if the SDF file is not in version 3.0 format. This is because some statements (even though they are not needed), which are supported in the old version but not in version 3.0, may be missed.

Timing Analysis Results

There is a wide range of timing analysis tools each with their own timing report format. The timing analysis results are typically in text format and difficult to understand and visualize.

The two main requirements for debugging timing analysis results is a gate-level design with a corresponding timing analysis results file. The various report formats can be converted with a utility to an XML format (e.g. *pt2Xml.pl* for PrimeTime). The results can then be loaded into the Verdi platform. After loading the design and timing analysis report in the Verdi platform, you have several options for using *nAnalyzer* to debug the timing analysis results:

- Sort and filter the timing results by start point, slack time or other specified options.
- Select any start/end point combination and display the results textually or schematically.
- Compare the timing analysis results with an equivalent SDF.

Use Models

Standard Delay Format (SDF)

The following list summarizes the currently supported SDF data:

1. IOPATH (cell delay)
2. INTERCONNECT (interconnect delay)

3. PATHCONSTRAINT (path delay)

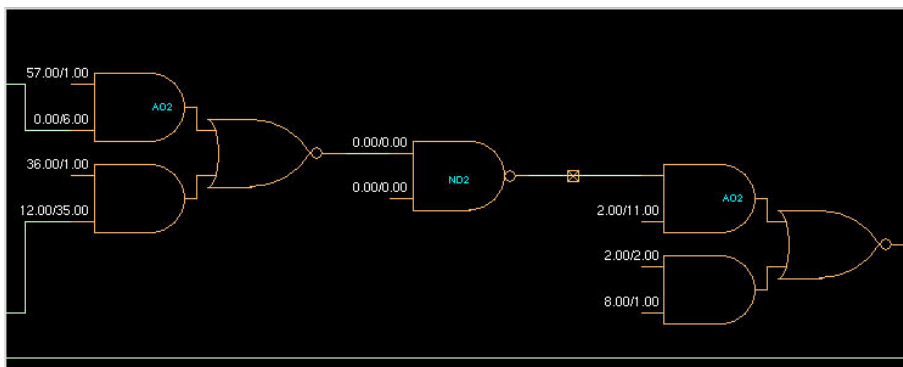
Other items are not supported.

The SDF data can be used in the following ways:

1. INTERCONNECT (interconnect delay)

If the imported SDF file contains INTERCONNECT information, *nSchema* will display interconnect delays in the schematics. To annotate the interconnect delay information onto the *nSchema* frame, you have to turn on the **Schematic -> SDF Annotation** option.

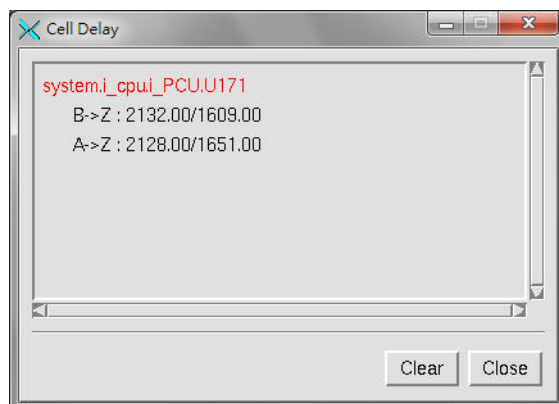
Figure 1 SDF Annotation in the nSchema Frame



2. IOPATH (cell delay)

If the imported SDF file contains IOPATH information, *nSchema* will obtain the information in the schematics. You may check the cell delay by clicking-right on the instance and select the **Cell Delay** command.

Figure 2 Show Cell Delay



3. PATHCONSTRAINT (path delay)

If the imported SDF file contains PATHCONSTRAINT information, *nAnalyzer* provides the command **Tools -> Browse/Watch/List -> SDF Path** from the main window to help you visualize and check the path constraints easily.

4. Shortest/Longest Path

With a SDF file imported, the timing debug functions in *nSchema* are enabled. You may invoke the **Trace -> Shortest/Longest Path** command or the **Trace -> Clock Skew** command in the *nSchema* frame to check the path delay based on IOPATH delay and INTERCONNECT delay.

Figure 3 Shortest/Longest Path Setup Form

The screenshot shows the 'Shortest/Longest Path' dialog box. It has a 'Trace Mode' section with 'Two Points', 'Fan-in', and 'Fan-out' options. Below is 'Transition Type' set to 'Both'. There are input fields for 'Delimiter' and 'Top Design'. The 'From/To Instance Port Pair' section contains two tables. The first table has 'From' set to 'system.i_cpu.i_PCU.IDR_reg[7]' and 'To' set to 'Q'. The second table has 'To' set to 'system.i_cpu.i_ALUB.IDR_reg[7]' and 'D'. At the bottom, there are checkboxes for 'Filter', 'Search All Paths', 'Search Range', 'Path Number' (set to 1), 'Stop on Tri-state', and 'Delay' fields. Buttons for 'Shortest Path', 'Longest Path', and 'Cancel' are at the bottom right.

If there are not any delay values displayed in *nSchema* after loading an SDF file which contains INTERCONNECT delay, please check the following:

1. Confirm the **Schematic -> SDF Annotation** option in the *nSchema* frame is turned *on*.
2. Change the value for the **Schematic -> Delay Type** option in the *nSchema* frame to confirm no values are displayed for any type (Minimum/Typical/Maximum).
3. Confirm the design hierarchy matches between the design and SDF file. If not, correct the design hierarchy of the SDF (in the loading menu) when you load the SDF file. (Also check hierarchy divider in SDF file and Load SDF form.) You can use the **File ->**

SDF -> Report Undefined Paths command in the main window to generate and view a list of undefined paths.

If the problem is not fixed by any of the above steps, please contact your local application engineer for technical support.

Timing Analysis Results

Utilities (Perl script) to convert timing analysis results are provided for the following tools:

- PrimeTime: pt2Xml.pl

Note:

Although a utility is provided for PrimeTime, the PrimeTime timing analysis report format can be directly read by the Verdi platform.

These timing report formats are available but are still in beta:

- Ambit: AMBIT.pl
- Astro: astro2Xml.pl
- Design Compiler: DC.pl
- IBM EinsTimer: EinsTimer.pl
- Magma: magma.pl
- Pearl: pearl.pl
- RTL Compiler: RTLC2Xml.pl
- Silicon Ensemble: SE2Xml.pl

The syntax for any of the provided Perl scripts is as follows:

```
perl_script -o <xml_filename> <report_filename>
```

After the timing analysis results are converted, you can load the results into the Verdi platform by choosing the **File -> Import Path Data File** command from the main window. In the *Import Timing Report* form, you would select the XML file name and click **OK**.

This opens the *Timing Report* form as shown below.

Figure 4 Timing Report Form

File: home1/allen_shieh/novas_demo_package_201107_0608_packed/verilog/gate/violate_path.rpt

Paths:

Configure Columns ☐ Attach Extracted Clock Domain Results Delay Precision 0.01

Start Point	End Point	Arrival Time	Required Time	Slack
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[0]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[1]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[2]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[3]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[4]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[5]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[6]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_ACC_tmp_reg[7]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[0]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[1]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[2]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[3]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[4]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[5]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[6]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_ALUB_IKR_tmp_reg[7]	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_CCUCO_reg	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_CCUCI_reg	0.00	24.00	(24.00)
system.reset (input port clocked by clock)	system_i_cpu_i_CCI1_C2_reg	0.00	24.00	(24.00)

Enable Sorting
Sort Slack Time by:
☐ Descending
☒ Ascending

Enable Grouping
Group by:
☐ Start Point
☐ Clock Domain of Start Point
☒ Clock Domain of End Point

Filter: Filter

Information:
 39: slack (NET) 24.00

Show on:
☒ File Viewer ☐ nSchema
☒ Path Report
☐ Slew Summary Report
 Delimiter:

Set False Path... Show Compare with SDF Close

In the report form, you can sort the results by slack time and/or group the results by the start point or start/end clock point. After selecting a result, the related report details are summarized in the **Information** box. Refer to the [Debug PrimeTime Results](#) tutorial for details on analyzing and viewing the loaded results.

Tutorials

The following sections are included:

- [Use SDF Data to Determine Longest Path](#)
- [Debug PrimeTime Results](#)

Before you begin the tutorials, follow the instructions in the [Before You Begin](#) section of the *Introduction* chapter.

Use SDF Data to Determine Longest Path

Load the SDF File

1. Change to the Verilog gate directory:

```
% cd <working_dir>/demo/verilog/gate
```

2. Confirm the symbol libraries are set as follows:

```
% setenv NOVAS_LIBPATHS $NOVAS_HOME/share/symlib/xx  
% setenv NOVAS_LIBS lsi10k_u
```

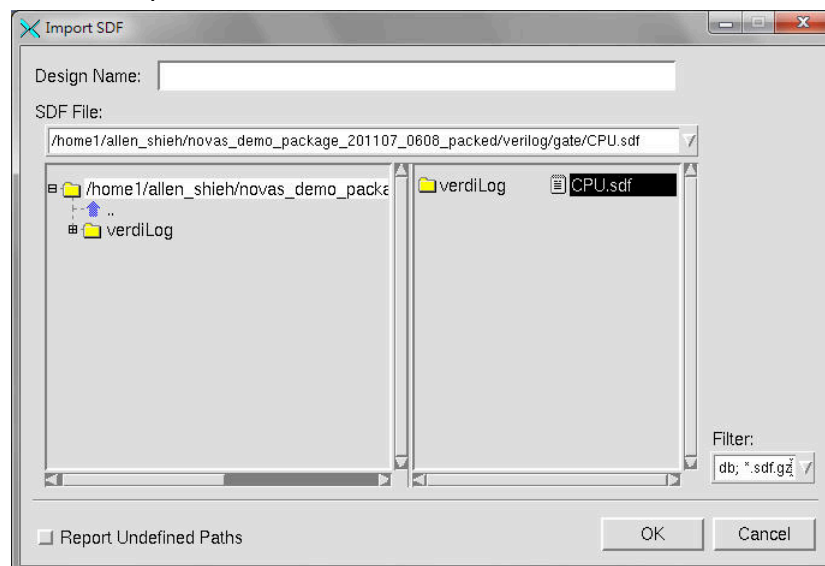
where xx will be either 32 or 64 depending on your system.

3. Load the gate-level design using the following command:

```
% verdi -f run.f &
```

4. In the main window, choose the **File -> SDF -> Load SDF Files** command to open the *Import SDF* form.

Figure 5 *Import SDF Form*



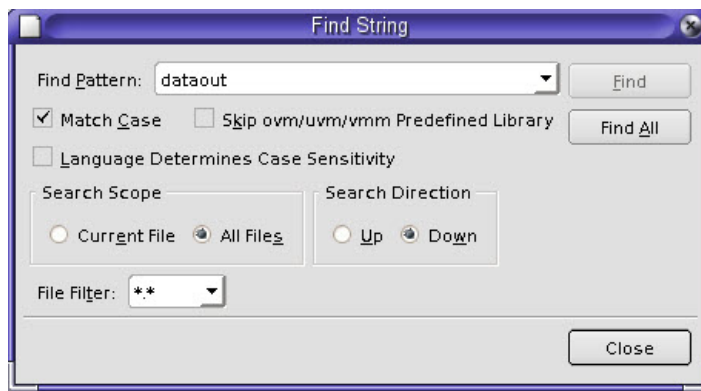
5. In the *Import SDF* form, select *CPU.sdf*.
6. Click the **OK** button.

Trace the Logic between Two Points

Before determining the longest path, let's identify the logic between *IDR_reg[7]* and *IXR_reg[7]*.

1. In the main window, choose the **Source -> Find String** command to open the *Find String* form.
2. In the *Find String* form, enter *IDR_reg* in the **Find Pattern** text field.
3. Select the **In All Files** option. The form will be similar to the following:

Figure 6 *Find String Form*

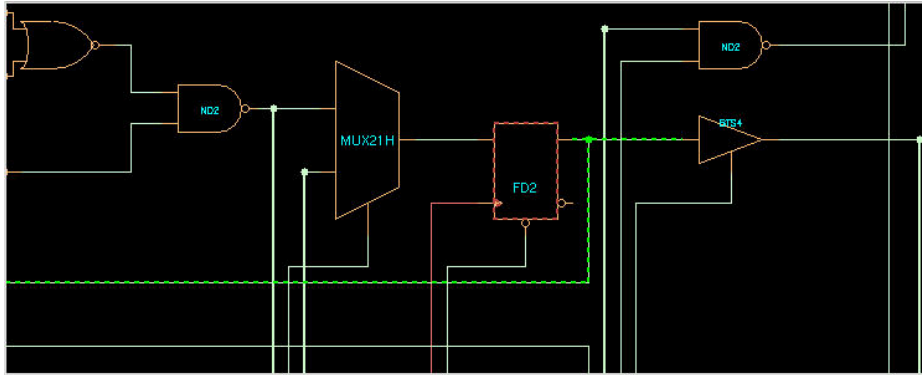


4. Click the **Find All** button.
5. In the *Message* frame, double-click bit 7, *IDR_reg[7]*. The line in the *Message* frame is similar to the following:

```
CPU.vg(113): FD2 \IDR_reg[7] ( .D(\IDR126[7] ), ...
```
6. In the source code frame, select the *\IDR_reg[7]* instance, invoke the right-click command menu and choose the **Drag** command.
7. Click the **New Schematic** icon on the toolbar to open the *nSchema* frame.
8. In the *nSchema* frame, enable the **Schematic -> Auto Fit Found Object(s)** command.
9. Invoke the right-click command menu in the *nSchema* frame and select the **Drop** command.

The instance will be highlighted, similar to the following:

Figure 7 *nSchema with Instance Highlighted*



10. In the *nSchema* frame, choose the **Trace -> Two Points** command to open the *Trace Two Points* form.
11. Select the Q output pin of instance `\IDR_reg[7]` and use the middle mouse button to drag and drop the pin to the **From** field of the *Trace Two Points* form.
12. In the main window, choose the **Source -> Find String** command to open the *Find String* form.
13. In the *Find String* form, enter `IXR_reg` in the **Find Pattern** text field.
14. Select the **In All Files** option.
15. Click the **Find All** button.
16. In the *Message* frame, double-click bit 7, `IXR_reg[7]`. The line in the message pane is similar to the following:

```
CPU.vg(743): FD2 \IXR_reg[7] ( .D(ALU[7] ), ...
```
17. In the source code frame, select the `\IXR_reg[7]` instance.
18. Use the middle mouse button to drag and drop selected instance from the source code frame to the **To** field of the *Trace Two Points* form. Confirm port *D* is selected in the *Trace Two Points* form.
19. In the *Trace Two Points* form, turn on the **Stop on FF/Latch**, **Stop on TriState** and **Create Window** options.

The form will be similar to the following:

Figure 8 Trace Two Points Form

Trace Mode

☒ Instance Port Based ☐ Signal Based

Delimiter: .

☐ Filter: Filter Top Design:

From/To Instance Port Pair

Instance Name	Port Name
system.i_cpu.i_PCU.IDR_reg[7]	Q

From: system.i_cpu.i_PCU.IDR_reg[7] Q

Instance Name	Port Name
system.i_cpu.i_ALUB.IDR_reg[7]	Q
system.i_cpu.i_ALUB.IDR_reg[7]	CD
system.i_cpu.i_ALUB.IDR_reg[7]	CP
system.i_cpu.i_ALUB.IDR_reg[7]	D

To: system.i_cpu.i_ALUB.IDR_reg[7] D

☐ Stop on Flip-flop/Latch Passthrough Port Type

☐ Stop on Tri-state

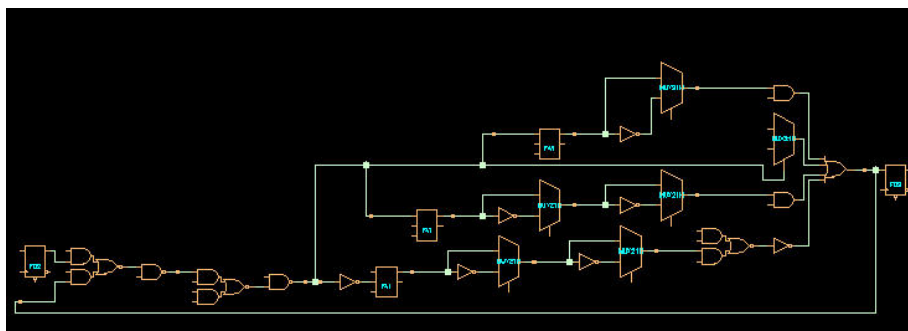
☐ Create Window

Trace Close

20. Click the **Trace** button.

The partial flattened schematic will be generated and displayed in *nSchema*, similar to the following:

Figure 9 Trace Two Points Schematic Results

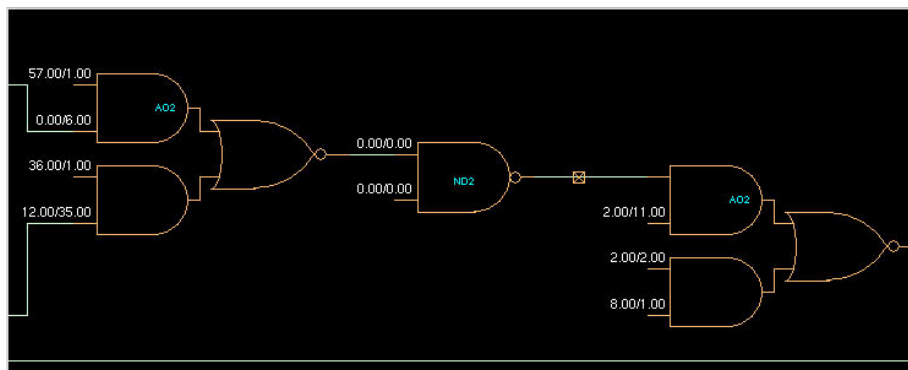


Calculate Longest Path

Now let's calculate the longest path between *IDR_reg[7]* and *IXR_reg[7]*.

1. Continue from the previous steps. The start and end points have been located and the SDF file is loaded.
2. In the *View Trace Results nSchema* frame, choose the **Schematic -> SDF Annotation** command.
3. Drag-left to zoom in around some logic and see the SDF annotation. The results will be similar to the following:

Figure 10 Schematic Excerpt with SDF Annotation



You can change the SDF delay type and scale by selecting different options under the **Schematic -> Delay Type** and **Schematic -> Delay Scale** commands.

4. In the *View Trace Result nSchema* frame, choose the **Trace -> Shortest/Longest Path** command to open the *Shortest/Longest Path* form.

The form will be similar to the following:

Figure 11 Shortest/Longest Path Form

The **From** and **To** fields are automatically filled in from the results of the Trace Two Points exercise. If you did not perform **Trace Two Points** first, you would need to follow similar steps to locate the start/end points and add them to the *Shortest/Longest Path* form.

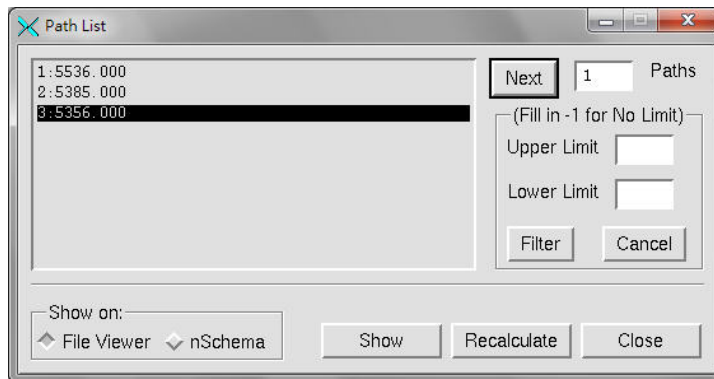
5. Leave all options in their default state and click the **Longest Path** button and click **Yes** on the opened *Question* form. The *Path List* form opens and shows the delay value of the longest path.

The first longest path has a delay value of 5536ps.

6. In the *Path List* form, click the **Next** button a couple of times to display the next longest paths of 5385ps and 5356ps.

The results will be similar to the following:

Figure 12 Path List Form

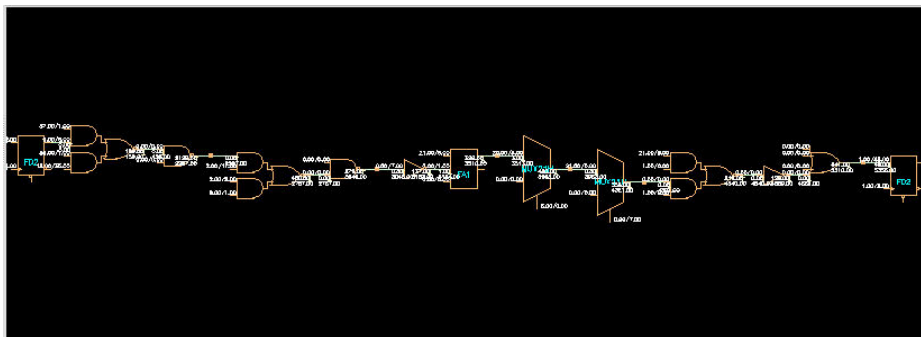


7. Select the 5356ps path.

8. In the **Show On** field, select the **nSchema** option and click **Show**.

A new *nSchema* frame will open displaying the logic in the path similar to the following:

Figure 13 Logic for 5310ps Delay Path



9. In the **Show On** field of the *Path List* form, select the **File Viewer** option and click **Show**.

A new *View Trace Result Text* frame will open displaying the logic in the path similar to the following:

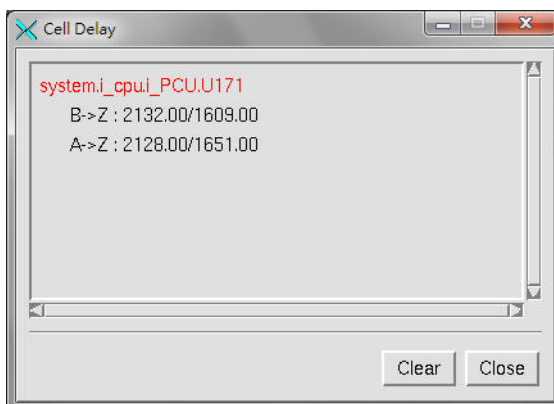
Figure 14 File Viewer Displaying 5310ps Delay Path

	TotalDelay	Delay	Type	Path
1				
2				
3	0.000	0.000	Rising	system.i_cpu.i_PCU.IDR_reg[7].Q->system.i_cpu.i_PCU.U204.B
4	159.000	159.000	Falling	system.i_cpu.i_PCU.U204.B->system.i_cpu.i_PCU.U204.Z
5	159.000	0.000	Falling	system.i_cpu.i_PCU.U204.Z->system.i_cpu.i_PCU.U171.A
6	2287.000	2128.000	Rising	system.i_cpu.i_PCU.U171.A->system.i_cpu.i_PCU.U171.Z
7	2287.000	0.000	Rising	system.i_cpu.i_PCU.U171.Z->system.i_ALUB.U279.A
8	2767.000	480.000	Falling	system.i_cpu.i_ALUB.U279.A->system.i_cpu.i_ALUB.U279.Z
9	2767.000	0.000	Falling	system.i_cpu.i_ALUB.U279.Z->system.i_cpu.i_ALUB.U252.B
10	3046.000	279.000	Rising	system.i_cpu.i_ALUB.U252.B->system.i_cpu.i_ALUB.U252.Z
11	3046.000	0.000	Rising	system.i_cpu.i_ALUB.U252.Z->system.i_cpu.i_ALUB.i_alu.sub_25_1.U8.A
12	3183.000	137.000	Falling	system.i_cpu.i_ALUB.i_alu.sub_25_1.U8.A->system.i_cpu.i_ALUB.i_alu.sub_25_1.U8
13	3184.000	1.000	Falling	system.i_cpu.i_ALUB.i_alu.sub_25_1.U8.Z->system.i_cpu.i_ALUB.i_alu.sub_25_1.U2
14	3510.000	326.000	Falling	system.i_cpu.i_ALUB.i_alu.sub_25_1.U2.7.B->system.i_cpu.i_ALUB.i_alu.sub_25_1
15	3513.000	3.000	Falling	system.i_cpu.i_ALUB.i_alu.sub_25_1.U2.7.S->system.i_cpu.i_ALUB.i_alu.sub_25_U1
16	3963.000	450.000	Falling	system.i_cpu.i_ALUB.i_alu.sub_25_U18.A->system.i_cpu.i_ALUB.i_alu.sub_25_U18.Z
17	3963.000	0.000	Falling	system.i_cpu.i_ALUB.i_alu.sub_25_U18.Z->system.i_cpu.i_ALUB.i_alu.add_25_U4.A
18	4321.000	358.000	Falling	system.i_cpu.i_ALUB.i_alu.add_25_U4.A->system.i_cpu.i_ALUB.i_alu.add_25_U4.Z

From the delay report shown on the *Text* frame, you can see the rising delay from U171.A to U171.Z is large. You want to locate where the instance U171 is in *nSchema*.

- Click the banner of the *View Trace Result Text* frame and drag to move it to another location, so that you can see the schematic and text views together.
- In the *View Trace Result Text* frame, select line 6 with “system.i_cpu.i_PCU.U171.A->system.i_cpu.i_PCU.U171.Z”.
- Use the middle mouse button to drag and drop this line from the *View Trace Result Text* frame to the *View Trace Result nSchema* frame. The instance will be highlighted.
- In the *nSchema* frame, right-click on the selected instance to open the right mouse button context menu and choose the **Cell Delay** command.

Figure 15 Show Cell Delay Form



You can see that for instance *U171*, from pin “A” to pin “Z” the rising delay is 2128ps, and falling delay is 1651ps. This helps to explain the delay you saw in the *File Viewer*.

14. In the *Cell Delay* and *Path List* windows, click the **Close** button.
15. Choose the **File -> Close** command in the *View Trace Result Text* and *nSchema* frames. You should leave one *nSchema* frame open.

At this time, in the open *nSchema* frame, you can choose the **Trace -> Shortest/Longest Path** command again and trace the longest path by **Rising** or **Falling** transition types or by specifying a search range and you’ll get different results. You could also investigate the shortest path between the two points.
16. When you’ve completed your investigation, exit the Verdi platform by choosing the **File -> Exit** command in the main window.

Debug PrimeTime Results

After running the PrimeTime STA tool, you get a timing report that shows paths that have not met timing constraints. You want to further investigate the failed paths.

Load the PrimeTime Results

The Verdi platform can convert a PrimeTime report to standard XML format.

1. Change to the Verilog gate directory:

```
% cd <working_dir>/demo/verilog/gate
```

2. Confirm the symbol libraries are set as follows:

```
% setenv NOVAS_LIBPATHS $NOVAS_HOME/share/symlib/xx
% setenv NOVAS_LIBS lsi10k_u
```

where xx will be either 32 or 64 depending on your system.

3. Compile the gate-level design using the following command:

```
% vericom -f run.f
```

4. Load the compiled design using the following command:

```
% verdi -lib work -top system &
```

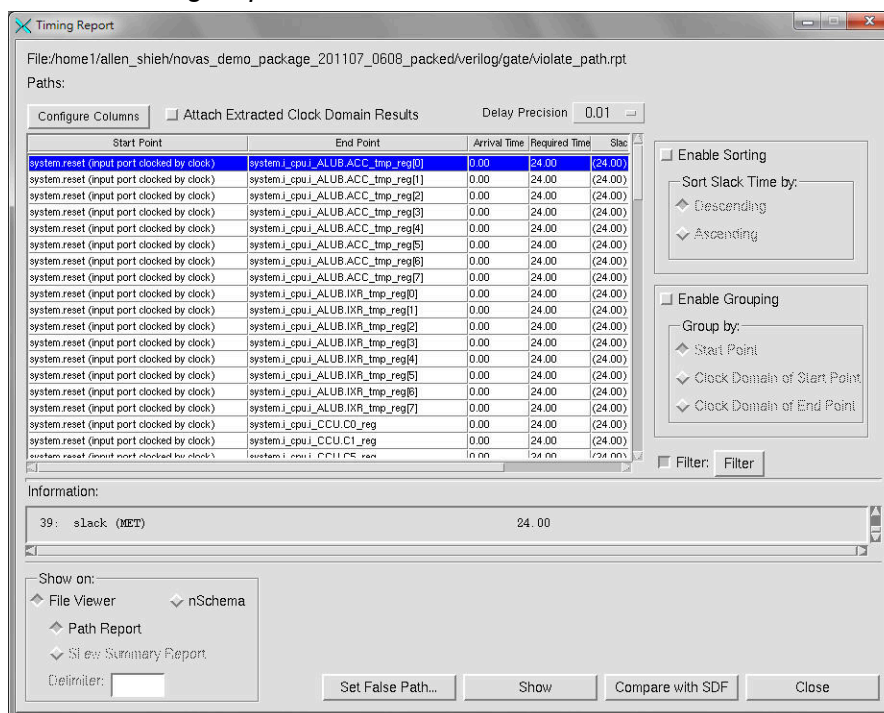
5. Execute the utility *pt2Xml.pl* **-o** *violate_path.xml* *violate_path.rpt* to create *violate_path.xml*.

Note:

This command is optional for PrimeTime. The PrimeTime timing analysis report can be read directly.

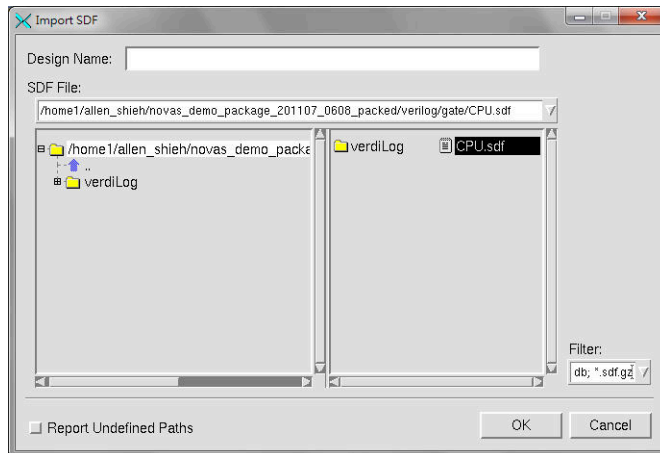
6. In the main window, choose the **File -> Import Path Data Files** command.
7. Since the timing results are from PrimeTime, you can load the results directly. Select the timing report file *violated_path.rpt*.
8. Click the **OK** button. The PrimeTime timing report displays in the *Timing Report* form, as shown below:

Figure 16 Timing Report Form



9. Choose the **File -> SDF -> Load SDF Files** command in the main window to open the *Import SDF* form.

Figure 17 Import SDF Form



10. Select *CPU.sdf*.
11. Click the **OK** button.

Show the Results on nSchema

1. In the *Timing Report* form, turn on the **Enable Sorting** option.
2. Select the **Ascending** option in the **Sort Slack Time By** field.
The red colored paths are timing violated paths.
3. Select the first red one. The slack time is -46.97 .
The information panel shows the cumulative timing along the path.

Figure 18 Sorted Timing Report

File: violate_path.rpt

Path:

Configure Columns... ☐ Attach Extracted Clock Domain Results Delay Precision: 0.01

	Start Point	End Point	Arrival Time	Required Time	Slack
1	system_i_cpu.i...	system_i_cpu.i...	95.62	48.65	(-46.97)
2	system_i_cpu.i...	system_i_cpu.i...	92.49	48.65	(-43.84)
3	system_i_cpu.i...	system_i_cpu.i...	92.49	48.65	(-43.84)
4	system_i_cpu.i...	system_i_cpu.i...	82.36	48.65	(-33.71)
5	system_i_cpu.i...	system_i_cpu.i...	82.36	48.65	(-33.71)
6	system_i_cpu.i...	system_i_cpu.i...	72.22	48.65	(-23.57)
7	system_i_cpu.i...	system_i_cpu.i...	72.22	48.65	(-23.57)
8	system_i_cpu.i...	system_i_cpu.i...	62.09	48.65	(-13.44)
9	system_i_cpu.i...	system_i_cpu.i...	62.09	48.65	(-13.44)
10	system_i_cpu.i...	system_i_cpu.i...	51.95	48.65	(-3.30)

Information:

☒ Enable Sorting
 Sort Slack Time By:
☐ Descending
☒ Ascending

☐ Enable Grouping
 Group by:
☒ Start Point
☐ Clock Domain of Start Point
☐ Clock Domain of End Point

☒ Filter

Show On
☒ File Viewer ☐ nSchema
☒ Path Report
☐ Skew Summary Report
 Delimiter: /

4. Select the **nSchema** option in the **Show On** field, and click the **Show** button to see the path in the *Prime Time Path nSchema* frame.
5. In the *Prime Time Path nSchema* frame, turn on the **View -> Instance Name** toggle command.
6. In the *Timing Report* form, select the **File Viewer** option in the **Show On** field, and click **Show** to see the path listing in the *View Trace Result Text* frame.

The worst case path with a slack of -46.97 is from *U235.Z* to *U101.Z*.

Figure 19 Path Results in the Text Frame

1	TotalDelay	Delay	Type	Path
2				
3	25.00	0.00	Falling	system.i_cpu.i_CCU.mux_sel_reg[2].CP->system.i_cpu.i_CCU.mux_sel_re
4	25.67	0.67	Rising	system.i_cpu.i_CCU.mux_sel_reg[2].Q->system.i_cpu.i_PCU.U235.Z
5	68.53	42.86	Rising	system.i_cpu.i_PCU.U235.Z->system.i_cpu.i_PCU.U193.Z
6	69.08	0.55	Falling	system.i_cpu.i_PCU.U193.Z->system.i_cpu.i_PCU.U204.Z
7	81.08	12.00	Rising	system.i_cpu.i_PCU.U204.Z->system.i_cpu.i_PCU.U171.Z
8	81.71	0.63	Falling	system.i_cpu.i_PCU.U171.Z->system.i_cpu.i_ALUB.U279.Z
9	81.93	0.22	Rising	system.i_cpu.i_ALUB.U279.Z->system.i_cpu.i_ALUB.U252.Z
10	82.60	0.67	Falling	system.i_cpu.i_ALUB.U252.Z->system.i_cpu.i_ALUB.i_alu.sub_25_1.U8.Z
11	88.74	6.14	Rising	system.i_cpu.i_ALUB.i_alu.sub_25_1.U8.Z->system.i_cpu.i_ALUB.i_alu
12	89.41	0.67	Falling	system.i_cpu.i_ALUB.i_alu.sub_25_1.U2_7.S->system.i_cpu.i_ALUB.i_al
13	90.62	1.21	Falling	system.i_cpu.i_ALUB.i_alu.sub_25.U26.Z->system.i_cpu.i_ALUB.i_alu.e
14	91.29	0.67	Rising	system.i_cpu.i_ALUB.i_alu.sub_25.U18.Z->system.i_cpu.i_ALUB.i_alu.i
15	92.50	1.21	Rising	system.i_cpu.i_ALUB.i_alu.add_25.U5.Z->system.i_cpu.i_ALUB.i_alu.ac
16	93.05	0.55	Falling	system.i_cpu.i_ALUB.i_alu.add_25.U4.Z->system.i_cpu.i_ALUB.i_alu.U1
17	93.72	0.67	Rising	system.i_cpu.i_ALUB.i_alu.U156.Z->system.i_cpu.i_ALUB.i_alu.U181.Z
18	95.62	1.90	Rising	system.i_cpu.i_ALUB.i_alu.U181.Z->system.i_cpu.i_ALUB.i_alu.U101.Z

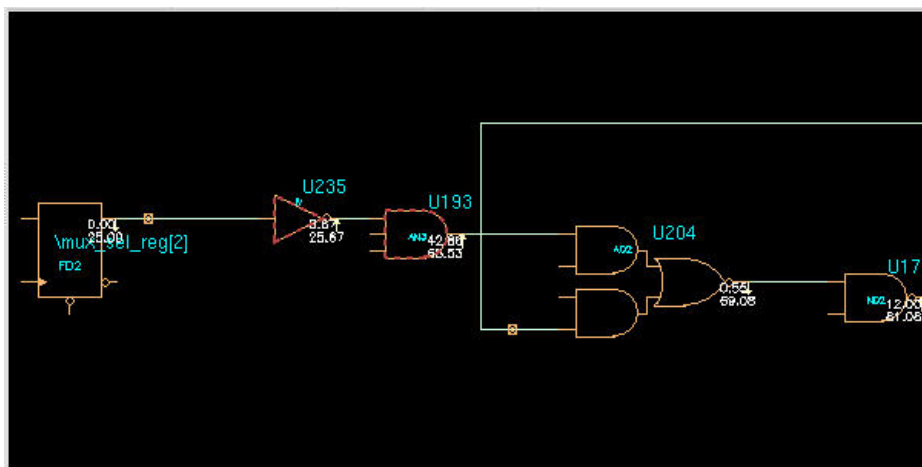
From the file viewer, notice the unusual delay value of 42.860 on line 5. This delay value is abnormal and needs to be looked into further.

- Click the banner of the *View Trace Result Text* frame and then drag to move it to another location, so that you can see the *nSchema* frame and the *Text* frame together.

- Drag and drop line 5 from the *Text* frame into the *nSchema* frame.

The problem is the delay on the output of *U193*.

Figure 20 Path Results in the nSchema Frame



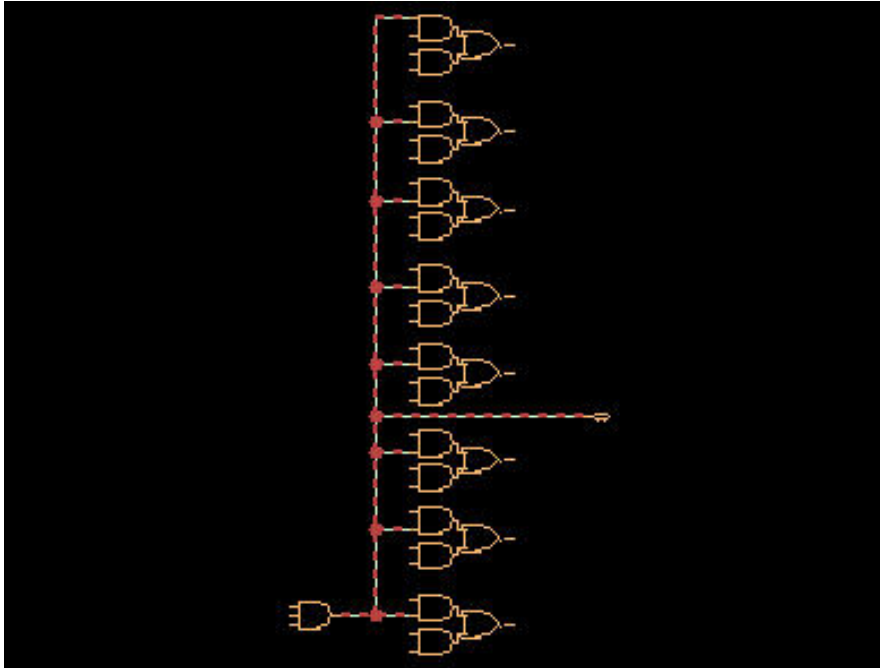
Isolate the Logic

- In the *nSchema* frame, turn on the **View -> Net Name** toggle command.
- Select *net n524*, connected to the output of *U193*.

3. Choose the **Tools -> New Schematic -> Connectivity** command.

The trace connectivity shows the loading structure. U193 is driving 8 cells, as shown below:

Figure 21 Trace Connectivity Results



In this situation, you can change *U193* to a cell with the same functionality but larger driving capability, or you can re-partition the structure to reduce the delay.

4. In the *nSchema* frame, enable the **Schematic -> SDF Annotation** option to look at the timing from layout.

After you decide on the change you want to make, you can use *nECO* to make the change from the current *nSchema* view. Refer to the *nECO User's Guide and Tutorial* for complete details.

Compare with SDF

You can also compare any of the timing paths with the SDF results in a text format.

1. In the *Timing Report* form, select the first red path with slack time -46.97.
2. Click the **Compare with SDF** button to open the *Compare with SDF* form.

3. In the *Compare with SDF* form, toggle to select **Default** for the **Delay Scale** field and **Typical** for the **Delay Type** field. The form will be similar to the following:

Figure 22 Compare with SDF Form

4. Click the **OK** button. The *View Trace Result Text* frame opens with the comparison results as a new tab in the same location as the previously opened *Text* frame, similar to the following:

Figure 23 View Trace Results Text Frame

	Delay	Type	Path
1			
2			
3	(PT) 0.000	Falling	system.i_cpu.i_CCU.mux_sel_reg[2].CP->system.i_cpu.i_CCU.mux_sel_reg[2].Q
4	(PT)-----Delay Sum: 0.000		
5	(SDF) 637.000	Falling	system.i_cpu.i_CCU.mux_sel_reg[2].CP->system.i_cpu.i_CCU.mux_sel_reg[2].Q
6	(SDF)-----Delay Sum: 637.000		
7			
8	(PT) 0.670	Rising	system.i_cpu.i_CCU.mux_sel_reg[2].Q->system.i_cpu.i_PCU.U235.Z
9	(PT)-----Delay Sum: 0.670		
10	(SDF) 12.000	Rising	system.i_cpu.i_CCU.mux_sel_reg[2].Q->system.i_cpu.i_PCU.U235.A
11	(SDF) 149.000	Rising	system.i_cpu.i_PCU.U235.A->system.i_cpu.i_PCU.U235.Z
12	(SDF)-----Delay Sum: 161.000		
13			
14	(PT) 42.860	Rising	system.i_cpu.i_PCU.U235.Z->system.i_cpu.i_PCU.U193.Z
15	(PT)-----Delay Sum: 42.860		
16	(SDF) 0.000	Rising	system.i_cpu.i_PCU.U235.Z->system.i_cpu.i_PCU.U193.A
17	(SDF) 516.000	Rising	system.i_cpu.i_PCU.U193.A->system.i_cpu.i_PCU.U193.Z
18	(SDF)-----Delay Sum: 516.000		
19			
20	(PT) 0.550	Falling	system.i_cpu.i_PCU.U193.Z->system.i_cpu.i_PCU.U204.Z
21	(PT)-----Delay Sum: 0.550		
22	(SDF) 1.000	Falling	system.i_cpu.i_PCU.U193.Z->system.i_cpu.i_PCU.U204.A
23	(SDF) 390.000	Falling	system.i_cpu.i_PCU.U204.A->system.i_cpu.i_PCU.U204.Z
24	(SDF)-----Delay Sum: 391.000		

The PrimeTime results are indicated with PT and the SDF results are indicated with SDF. Each from/to point is listed with the respective sources results.

Attach Clock Domain Information

You can also attach the clock domain information to the timing paths. You need to extract clock domains first. Refer to *Clock/Reset Tree Analysis* chapter for background information.

1. In the main window, choose the **Tools -> Clock Analyzer -> Extract Clock Information** command to open the *Clock Extraction Settings* form.
2. In the *Clock Extraction Settings* form, click the **Import** button.
3. In the *Import* form, select **SDC** in the **Format** field and confirm the top design is *system* in the **Top Design** field, and select the *sdcsystem.sdc* file.
4. Click the **OK** button on the *Load SDC* form.
5. Click the **OK** button in the *Clock Extraction Settings* form. The *Clock Domain* frame opens in the same location as the *Message* frame.
6. In the *Timing Report* form, enable the **Attach Extracted Clock Domain Results** option and the clock information (**Clock Source of Start Point** and **Clock Source of End Point** columns) will be added to the table.

You can click the **Configure Columns** button to change the order of the column display and make it easier to see the results. You can also drag left on the column headers to change the width. The results will be similar to the following:

Figure 24 Attach Clock Results

Configure Columns		Attach Extracted Clock Domain Results		Delay Precision 0.01	
Start Point	End Point	Slack	Clock Source of Start Point	Clock Source of End Point	
(Domain 1) system_i_cpu_i_CCU.mux_3	(Domain 2) system_i_cpu_i_ALUB.II	(-46.97)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.A	(-43.84)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.II	(-43.84)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.A	(-33.71)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.II	(-33.71)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.A	(-23.57)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.II	(-23.57)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.II	(-13.44)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.A	(-13.44)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.A	(-3.30)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.II	(-3.30)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.A	(9.73)	system.clock	system.clock	
(Domain 1) system_i_cpu_i_CCU.bus_r1	(Domain 2) system_i_cpu_i_ALUB.II	(9.73)	system.clock	system.clock	
system.reset (input port clocked by cl0)	(Domain 1) system_i_cpu_i_CCU.ne	(13.58)		system.clock	
system.reset (input port clocked by cl0)	(Domain 1) system_i_cpu_i_CCU.ne	(14.50)		system.clock	
system.reset (input port clocked by cl0)	(Domain 1) system_i_cpu_i_CCU.ne	(15.41)		system.clock	
system.reset (input port clocked by cl0)	(Domain 1) system_i_cpu_i_CCU.ne	(16.32)		system.clock	
system.reset (input port clocked by cl0)	(Domain 1) system_i_cpu_i_CCU.ne	(17.23)		system.clock	
system.reset (input port clocked by cl0)	(Domain 1) system_i_cpu_i_CCU.ne	(18.15)		system.clock	

7. When you are finished with the investigation, exit the Verdi platform by invoking the **File -> Exit** command in the main window.

3

Clock/Reset Tree Analysis

This section describes the following topics:

- [Overview](#)
- [Clock Domain Analysis Background](#)
- [Tutorials](#)

Overview

In the area of clock/reset tree analysis, *nAnalyzer* supports two primary functions: viewing clock/reset trees in the schematic view and analyzing the clock domains in the design.

Clock/Reset Tree

Because the complexity of the clock/reset tree is increasing in VLSI designs, implementing an ideal clock/reset tree has become an important task. In most designs, the clock and reset trees have heavy fan-out loads. If the loads are not balanced among each branch, clock/reset skew may occur.

The main requirement for analyzing clock or reset trees is a gate-level design (you can also use an RTL design). You may also have a SDF file. After loading this information in the Verdi platform, you have several options for using the *nAnalyzer* module to view clock/reset trees:

- Extract the clock/reset tree and display the results in a compressed register view in the *nSchema* frame.
- Use the compressed register views to understand the polarity of the clock/reset tree.
- Expand the compressed register symbol to view the flip-flops or latches connected to the net.
- Highlight the clock/reset trees for easier viewing across schematic views.
- Analyze the clock/reset skew between branches if you have SDF information.

Clock Domain Analysis

A complex chip design may consist of several clock domains and analyzing the different domains is critical. You need to understand how many clock domains exist, how many gates are connected to each domain and determine if any paths cross clock domains.

The main requirement for analyzing clock domains is a gate-level design. You may also have a Synopsys Design Constraint (SDC) file. After loading this information in the *Verdi platform*, you have several options for using *nAnalyze* to understand your clock domains:

- Specify known clock sources or known constants to direct the analysis. The SDC file can be used if you have one.
- Extract the clock domains and determine whether the domains are resolved or not. See the *Clock Domain Analysis Background* section for more details.
- Analyze the clock domains for crossing paths and indicate whether a synchronizer is included or not.
- Display a selected clock domain or crossing path result textually or schematically.
- Highlight the clock domains or crossing paths for easier viewing across schematic frames.
- Identify the clock source for a selected flip-flop or latch.

Clock Domain Analysis Background

This section includes the following topics:

- [Terminology](#)
- [Extract Clock Source Tree](#)
- [Extract Clock Domain](#)
- [Build Clock Source Tree](#)
- [Mix-Clock Domain Crossing-Path](#)

Terminology

Clock Signal

A clock signal is a signal directly connected to the clock pin of a storage element in the design. The storage element could be flip-flop, latch or other memory storage element.

Internal Clock Source

The internal clock source is the signal closest to the primary input port driving the clock signals by passing through design hierarchy and the optional logic. The type of logic passed through is variable. In general, the logic could be a buffer, inverter, reduced buffer or inverter after constant value propagation, etc.

Clock Domain

A clock domain consists of groups of storage elements. Under an internal clock source, there might be two clock domains. One is triggered by the positive edge of the internal clock source. The other is triggered by the negative edge of the internal clock source. All the flip-flops, latches or other storage elements in a clock domain are triggered by the same edge of the internal clock source.

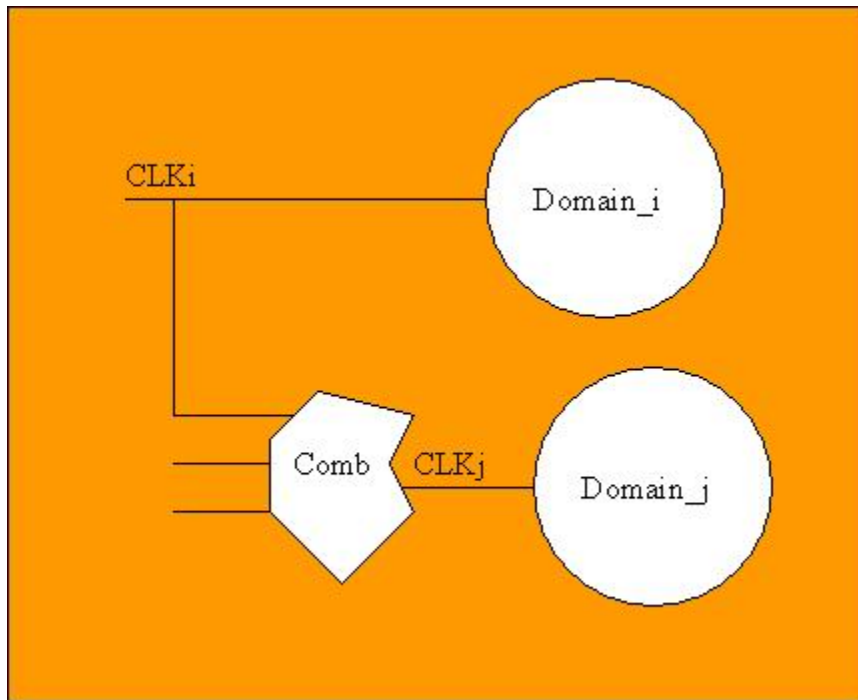
Clock Source Tree

The clock source tree is a tree structure. The tree node is an internal clock source. If a clock source is derived from another internal clock source by passing through some logic, it is child node of the internal clock source that drives it in the tree.

There are two ways for an internal clock source to be derived from its parent.

If the internal clock source is derived from its parent by passing through combinational logic, the derived type relative to its parent is *Level0*. This typically means it is a gated clock, as demonstrated in the following figure.

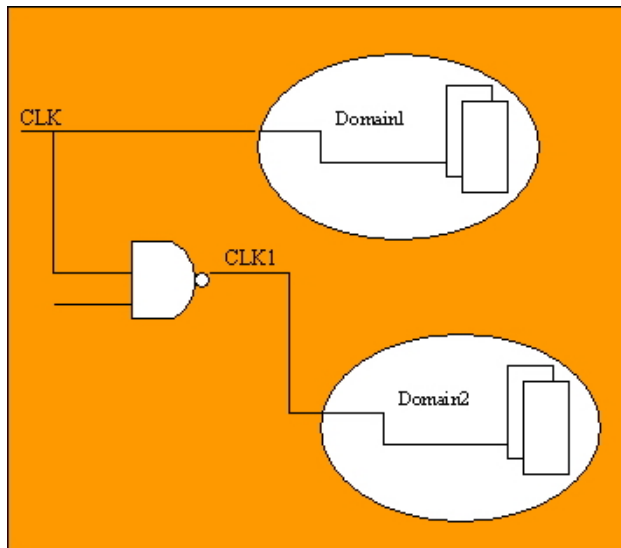
Figure 25 Parent Clock Source and Child Clock Source with Level0



In the above figure, there are two internal clock sources, CLK_i with clock domain $Domain_i$ and CLK_j with clock domain $Domain_j$. $Comb$ represents combinational logic other than a buffer or inverter. The internal clock source CLK_j is generated by CLK_i by passing through combinational logic. So, internal clock source CLK_j is the child of internal clock source CLK_i in the clock source tree. A *Level0* type is used to indicate the derived type of the internal clock source CLK_j relative to its parent.

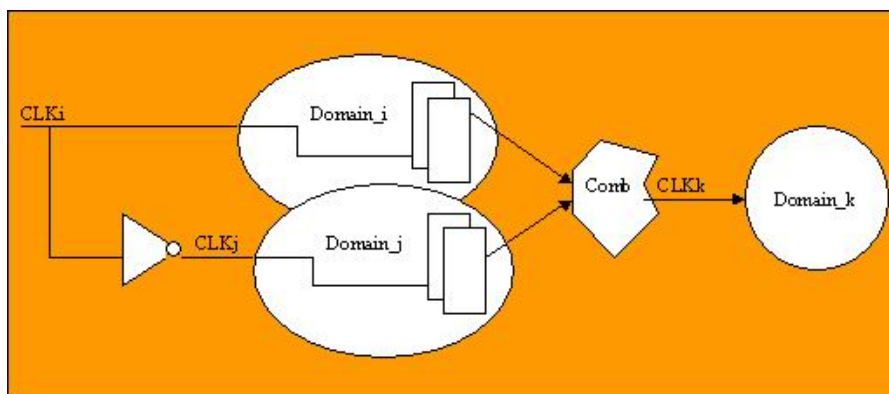
In addition to the derived type, another property is the phase shift of the internal clock source relative to its parent. For a *Level0* type child internal clock source, the phase difference to the parent internal clock source should be extracted. In the following figure, clock source CLK_1 has $T/2$ phase shift compared to parent internal clock source CLK .

Figure 26 Phase Shift between Clock Sources



If the internal clock source is generated from the combinational output of some flip-flops, which are triggered by the parent internal clock source, the derived type relative to its parent is *LevelN*. This typically means it is a generated clock, as demonstrated in the following figure.

Figure 27 Parent Clock Source and Child Clock Source with *LevelN*



In the above figure, there are three internal clock sources, CLK_i with clock domain $Domain_i$, rising edge triggered, and CLK_j with clock domain $Domain_j$, falling edge triggered. The internal clock source CLK_k is generated by combinational logic whose inputs are the output of flip-flops in $Domain_i$ or $Domain_j$. The internal clock source CLK_i does not pass into the combinational logic to generate the clock source CLK_k directly. Thus in the clock source tree, clock source CLK_k is child of clock source CLK_i and a *LevelN* type is used to indicate the derived type of the internal clock source CLK_k relative to its parent.

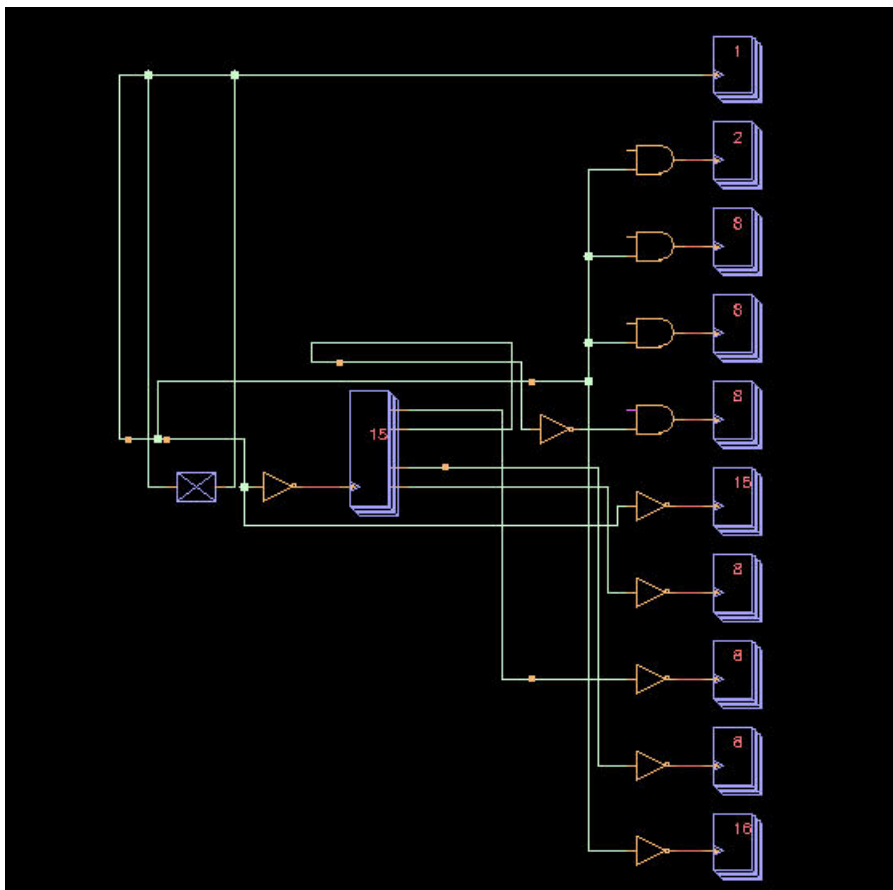
For *Level/N* derived types, the phase shift of the internal clock source relative to its parent can not be extracted automatically.

Clock Source

The root of the clock source tree is the clock source. A resolved clock source can be a primary input, a user specified clock source or the output of a register. Other sources such as a floating wire or the output of combinational logic are un-resolved clock sources. If more than one clock source drives an internal clock source, it is treated as multiple resolved. Multiple resolved internal clock sources will be the root of a clock source tree.

The following shows an example for the definition of clock sources. Assume the signal *system.clock* is the specified clock source.

Figure 28 Example *system.clock*



The clock domains are extracted as follows.

Figure 29 Extracted Clocks Domains for the Example

Clock Source Tree	Number (Total:2)	Number of Flip-flop (Total:105)	Resolve Type
system.clock	1	Total: 97	
system.clock (f)	1.f	46	Unresolved (Combinational Logic)
system.clock (r)	1.r	19	Unresolved (Combinational Logic)
system.i_cpu.i_CCU.C1	1.1		
system.i_cpu.i_CCU.C1 (r)	1.1.r	8	Resolved (Register Output)
system.i_cpu.i_CCU.C19	1.2		
system.i_cpu.i_CCU.C19 (f)	1.2.f	8	Resolved (Register Output)
system.i_cpu.i_CCU.C5	1.3		
system.i_cpu.i_CCU.C5 (f)	1.3.f	8	Resolved (Register Output)
system.i_cpu.i_CCU.C6	1.4		
system.i_cpu.i_CCU.C6 (f)	1.4.f	8	Resolved (Register Output)
system.i_cpu.i_ALUB.S1	2	Total: 8	
system.i_cpu.i_ALUB.S1 (f)	2.f	8	Unresolved (Combinational Logic)

The signal `system.clock` is specified as a clock source, so it is a root of the clock source tree. The flip-flops (FFs) under this domain have been marked as domain 1.f and 1.r where 'f' represents FFs driven by the falling edge and 'r' means rising edge.

A derived clock source, `system.i_cpu.i_CCU.C1` resides under the root source `system.clock`. `system.i_cpu.i_CCU.C1` was generated from a FF output so the domains under this source are identified as 1.1, which represents the sub-domains belonging to the root domain 1.

Extract Clock Source Tree

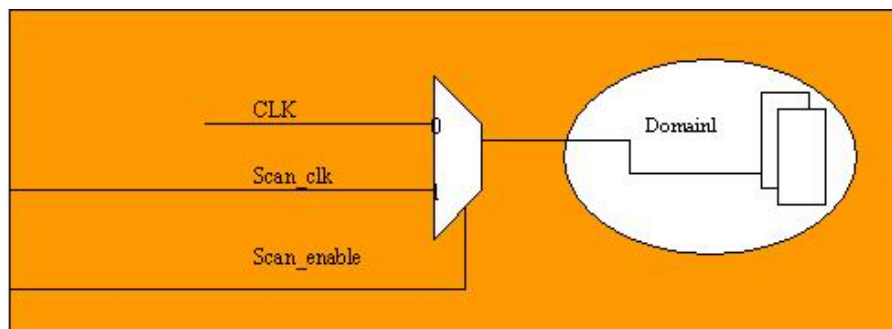
The clock analyzer is used as the engine to extract the clock source tree. There are two primary steps in the engine:

- Extract clock domains with internal clock source.
- Build the clock source tree based on the clock domain and extracted internal clock source.

Constant Value Propagation

Before any steps, the design can be reduced using constant value propagation.

Figure 30 Example for Constant Value Propagation



In the above figure, if *scan_enable* is specified as 0, the MUX will be reduced to a buffer (or inverter) with only one input *CLK*. If *scan_enable* is specified as 1, the MUX will be reduced to a buffer with only one input *scan_clk*.

Extract Clock Domain

The engine will collect all flip-flops (and latches) and find their clock signal. All the flip-flops (and latches) from the same clock signal are merged into a group. Then the engine will trace backward to find the internal clock source from the clock signals.

After the engine finds the internal clock source, all groups of flip-flops with the same internal clock source will be merged into at most two clock domains under the internal clock source. One domain will be triggered by the positive edge of the extracted internal clock source. The other domain will be triggered by the negative edge of the extracted internal clock source.

When finding the internal clock source of the flip-flop (and latches) groups, there are some options to guide the engine to pass through certain types of logic.

Pass Through Logic

When finding the internal clock source of groups of flip-flops (and latches), some logic can be passed through just like a wire connection.

- Pass through assignment.
- Pass through buffer.
- Pass through reduced buffer. Here *reduced* means that the logic is reduced by constant value setting.
- Pass through inverters.
- Pass through a specific gated clock cell. Here the specific gated clock cell means a library cell which has an input clock pin and output clock pin. The cell is designed to be used specifically as the gated cell for gated clocks to reduce the power consumption and reduce timing problems.
- Pass through cell with one input port and one output port.
- Pass through cell with one input port after specifying non-concerned signal.

Build Clock Source Tree

After extracting the clock domain, the engine will use the results to build the clock source tree, to find the clock sources and construct the parent-child relationship of all internal clock sources under the same clock source.

For each internal clock source signal, the engine will trace its fan in cone. When tracing, it will stop at:

- primary inputs,
- floating wires or output ports of unknown logic,
- flip-flop outputs,
- and specified clock sources.

In the path to the fan in cone, there may be some internal clock sources extracted during the first step. The engine will construct the parent-child relationship according to the derived type and extract the phase shift between them.

After the root of the clock source tree is found, the clock source is resolved if the clock source is a:

- primary input,
- flip-flop output,
- or specified clock source.

The clock source is unresolved if it is a:

- floating wire
- or output port of complex (or unknown) logic.

The clock source is multiple resolved if it is driven by more than one clock source.

Specify Clock Source

When building the clock source tree, you can specify a clock source to guide the engine. It is not a requirement but it may help the engine to determine the real clock source when more than one input is encountered for a cell.

The primary port or any signal in the design can be specified as a clock source with full hierarchy name. The specified clock source will be a root of a clock source tree. This allows you to treat some special clock sources different than others under the same root.

Mix-Clock Domain Crossing-Path

A mix-clock domain path crosses from the start point of one clock domain to the end point of another clock domain.

The following two figures show two different mix-clock domain paths.

Figure 31 Mix-Clock Domain Path

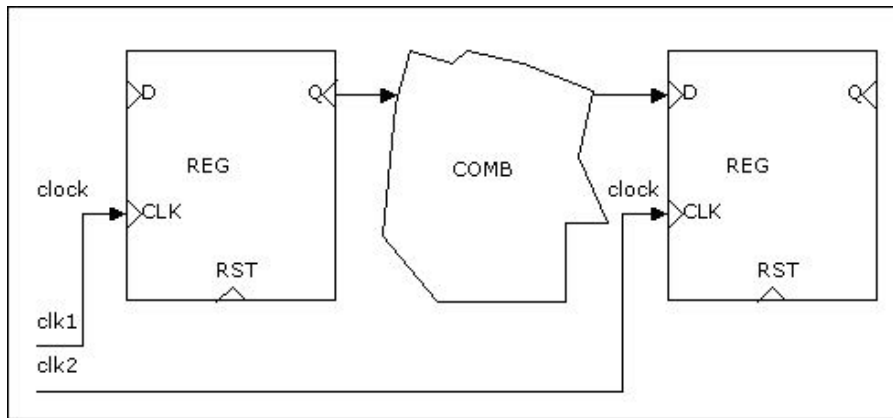
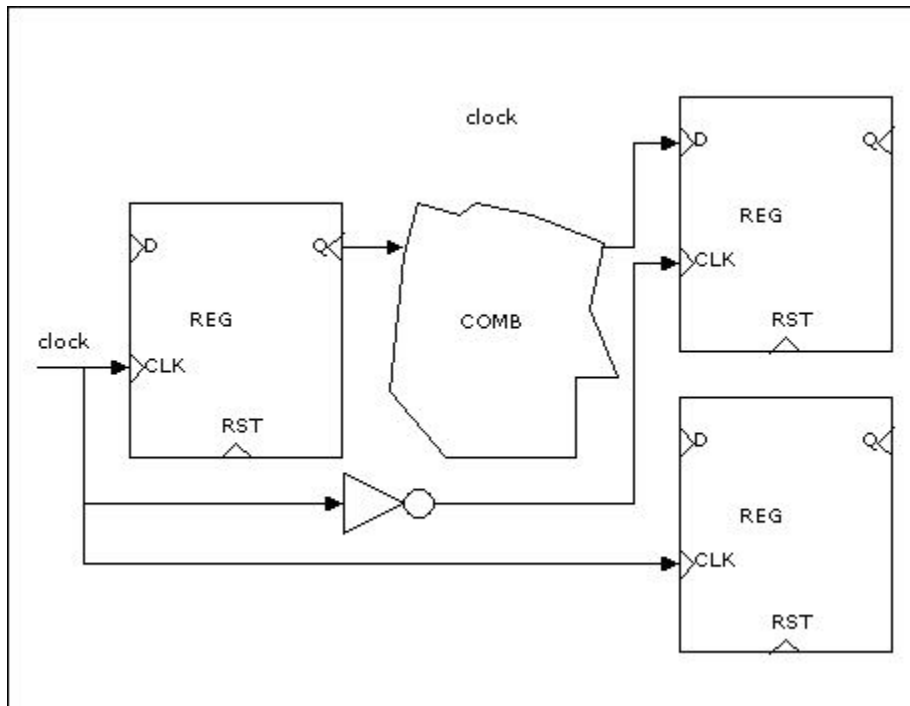


Figure 32 Mix-clock Domain Path 2



Here the clock domain is a general concept. You may have different interests in different applications. For example, if you are concerned with a timing problem, in the above clock source tree analysis, you can find that the clock source in the same clock source tree may have fixed frequency and phase relationship. Based on this, the timing in the clock domains with clock sources in the same clock source tree can be analyzed accurately. Thus, you can focus on the crossing path between different clock domains with clock sources in different clock source trees.

In extracting mix-clock domain paths, generally, *from domain* and *to domain* pairs should be specified to be applied to the extraction. Both the *from domain* and *to domain* could be a root of the tree, an internal clock source in the tree or a clock domain in the tree.

No matter how the *from domain* and *to domain* is specified, after extraction, the crossing path is listed under the *from* and *to* domain pairs with detailed path information from the start flip-flop to the end flip-flop. The specifying of the root of the tree and internal clock source will be expanded as all the clock domains under the specified node.

Tutorials

The following sections are included:

- [Display a Clock Tree](#)
- [Analyze Clock Skew](#)
- [Display a Reset Tree](#)
- [Understand and Debug Clock Domains](#)
- [Analyze Clock Domains for Crossing Paths](#)
- [Create a User-Defined Synchronizer](#)

Before you begin the tutorials, follow the instructions in the [Before You Begin](#) section of the *Introduction* chapter.

Display a Clock Tree

From within an *nSchema* or source code frame, you can display a clock tree (using a gate design) in a separate *nSchema* frame using the following steps:

1. Change to the Verilog gate demo directory:

```
% cd <working_dir>/demo/verilog/gate
```

2. Confirm the symbol libraries are set as follows:

```
% setenv NOVAS_LIBPATHS $VERDI_HOME/share/symlib/xx
% setenv NOVAS_LIBS lsi10k_u
```

where xx will be either 32 or 64 depending on your system.

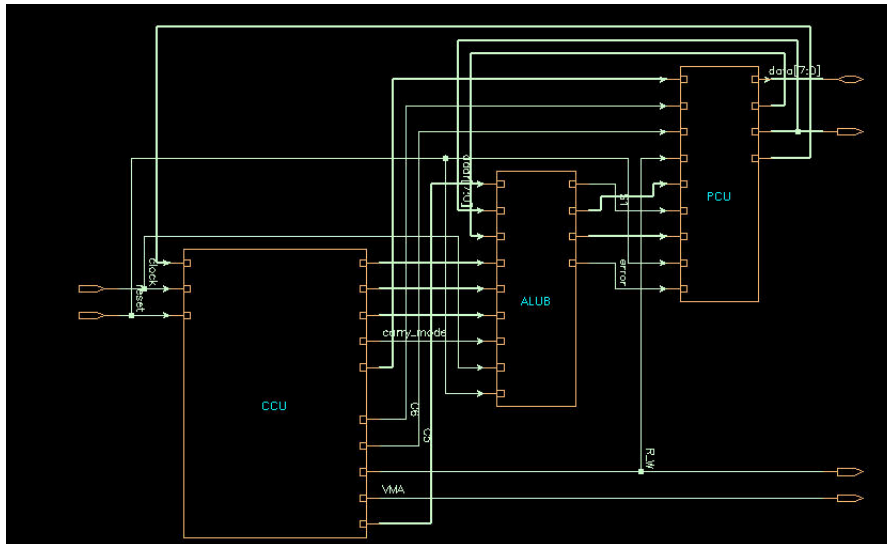
3. Invoke the Verdi platform:

```
% verdi -f run.f
```

4. Open the *i_cpu* instance in a new *nSchema* frame.

- a. Double-click the *i_cpu* instance in the <Inst._Tree> tab of the design browser frame to display the source code.
 - b. Click the **New Schematic** icon in the toolbar.
5. In the *nSchema* frame, choose the **View -> Net Name** command. The schematic is updated to display net names, as shown below:

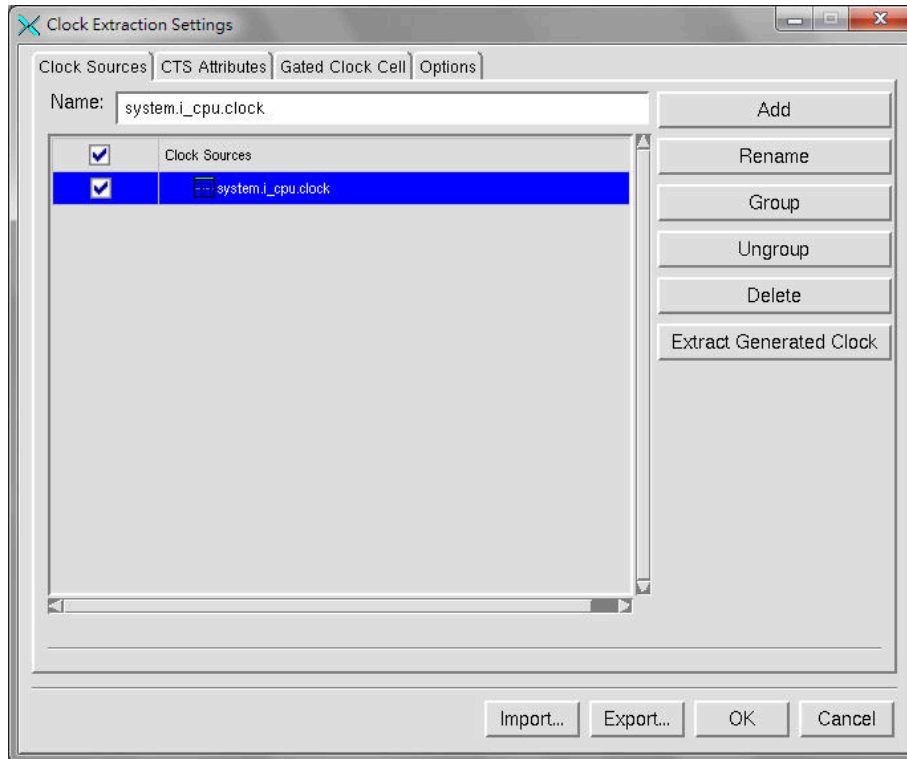
Figure 33 *nSchema Window Displaying Net Names*



6. In the *nSchema* frame, click to highlight and select the *clock* signal.
7. Choose the **Tools -> New Schematic -> Clock Tree** command.
A *Clock Extraction Settings* form opens.
8. Confirm the clock signal has been added to the **Clock Sources** section in the **Clock Sources** tab.
9. Turn *on* the **Show Details of Specified Domains (Clock Tree)** option in the **Options** tab.

The form will be similar to the following:

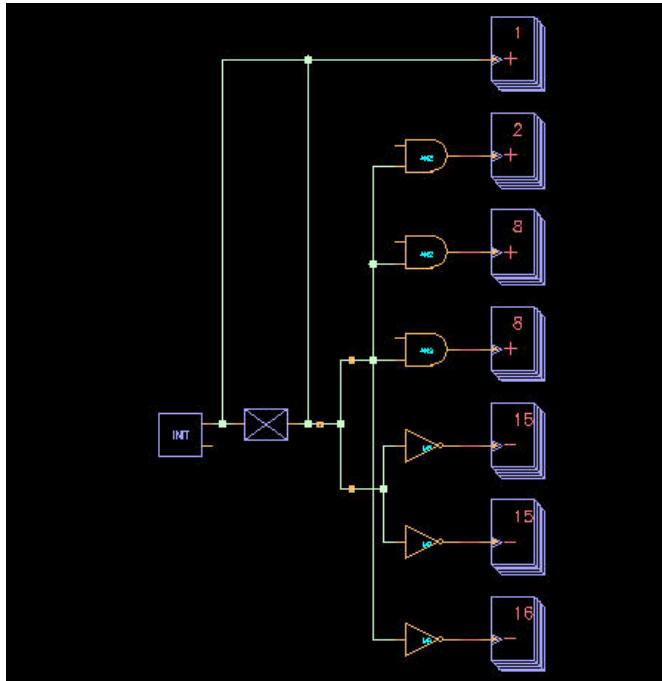
Figure 34 *Clock Extraction Settings Form*



10. Click the **OK** button.

A new *nSchema* frame is opened, as shown below:

Figure 35 *nSchema View Trace Clock Tree Results*



The primary clock fans out to several gated clocks. Each sub-clock drives one or more storage elements on the positive (indicated by a '+' symbol) or negative (indicated by a '-' symbol) edge.

A *Clock Tree Browser* frame is also opened as a new tab in the same location as the *Message* frame.

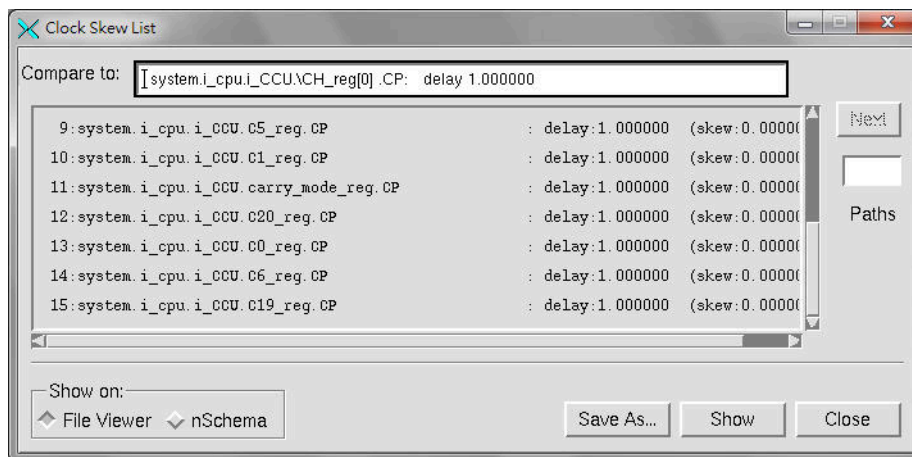
11. Double-click a compressed register view to expand the associated gates.
12. Select one of the expanded gates, click-right to open the right mouse button context menu and choose the **Collect Flip-Flop** command to return to a compressed register view.
13. Select a compressed register view symbol, click-right to open the right mouse button context menu and choose the **Black Box Information** command to display the associated gates as a text summary.
14. In the *Flip-Flop List* form, click the **Close** button.

Analyze Clock Skew

After the clock tree (or reset tree) has been extracted, you can load the SDF information and analyze the clock skew.

1. Continue from the previous example.
2. Load the SDF file.
 - a. In the main window, choose the **File -> SDF -> Load SDF Files** command.
 - b. In the *Import SDF* form, select *CPU.sdf* and click the **OK** button.
3. In the *View Trace Clock Tree Results* schematic frame, choose the **View -> Net Name** command.
4. Select net *n662* (second net up from the bottom).
5. Choose the **Trace -> Clock Tree -> Clock Skew** command.
6. In the *Clock Skew* form, turn on the **Longest** option in the **Compare With** field and turn on the **Show Maximum Skew** option.
7. Click the **Trace** button. The *Clock Skew List* form will open, similar to the following example:

Figure 36 Clock Skew List Form



8. In the *Clock Skew List* form, select line 10.
9. Select **File Viewer** in the **Show On** field and click the **Show** button. This summarizes the delay along the path textually and opens a new tab in the same location as the source code frame.

10. Select **nSchema** in the **Show On** field and click the **Show** button. You will see the trace result of the previous step in a schematic representation. The result will be shown in a new tab in the same location as the source code frame.
11. Close the Verdi platform by choosing the **File -> Exit** command from the main window.

Display a Reset Tree

From within the *nSchema* or *nTrace* frame, you can display the results of a reset tree in a separate window.

1. Change to the Verilog gate demo directory:

```
% cd <working_dir>/demo/verilog/gate
```

2. Confirm the symbol libraries are set as follows:

```
% setenv NOVAS_LIBPATHS $VERDI_HOME/share/symlib/xx  
% setenv NOVAS_LIBS lsi10k_u
```

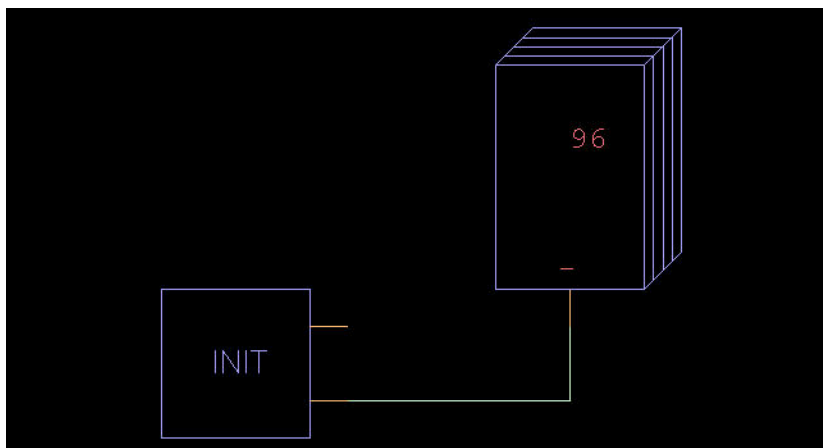
where xx will be either 32 or 64 depending on your system.

3. Invoke the Verdi platform:

```
% verdi -f run.f
```

4. In the **Inst._Tree** tab of the design browser frame, double-click the *i_cpu* block.
5. In the source code frame, select to highlight *reset*.
6. Choose the **Tools -> New Schematic from Source -> Reset Tree** command from the main window. An *nSchema* frame similar to the following figure opens:

Figure 37 View Trace Reset Tree Result



After the reset tree is displayed in the *nSchema* frame, you can expand any of the compressed register views or use SDF information to analyze reset skew.

7. Close the Verdi platform by choosing the **File -> Exit** command from the main window.

Understand and Debug Clock Domains

This tutorial will guide you through using the Verdi platform to extract your clock domains and check for crossing paths.

Extract Clock Domains

1. Change to the Verilog gate directory:

```
% cd <working_dir>/demo/verilog/gate
```

2. Confirm the symbol libraries are set as follows:

```
% setenv NOVAS_LIBPATHS $VERDI_HOME/share/symlib/xx
% setenv NOVAS_LIBS lsi10k_u
```

where xx will be either 32 or 64 depending on your system.

3. Compile the gate-level design using the following command:

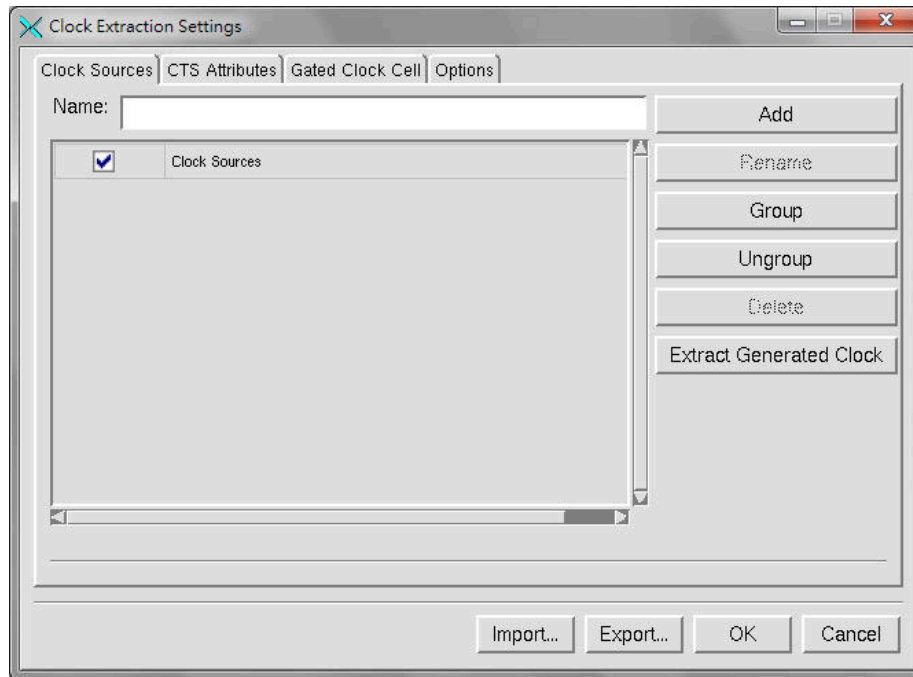
```
% vericom -f run.f
```

4. Load the compiled design using the following command:

```
% verdi -lib work -top system &
```

5. In the **Inst._Tree** tab of the design browser frame, select *system*.
6. From the main window, choose the **Tools -> Clock Analyzer -> Extract Clock Information** command to open the *Clock Extraction Settings* form. The form will be similar to the following figure.

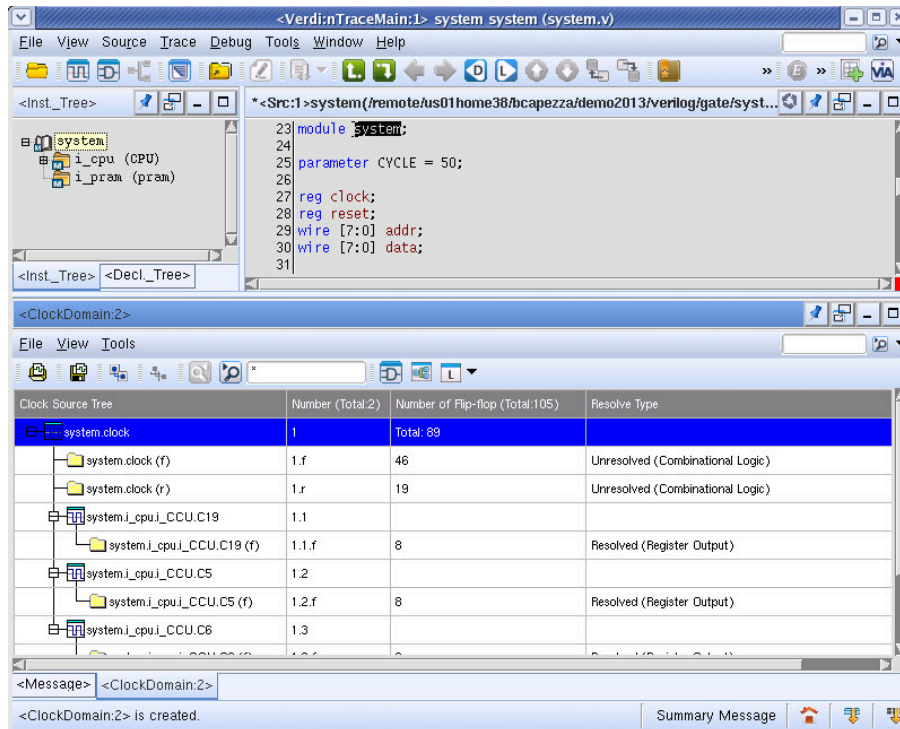
Figure 38 Clock Extraction Settings Form



7. Click the **OK** button. The *Clock Domain* frame will be opened in the same location as the *Message* frame.

Adjust the frame to have a better view for the *Clock Domain* frame, as shown in the following figure.

Figure 39 Extracted Clock Domain Results



The *Clock Domain* frame lists all the clock domains in the design. The results list the clock sources in a tree as rising or falling edge trigger, assigns a number to the top clock domain and sub-clock domains, identifies the number of flip-flops (FFs) in that clock domain, and indicates whether that domain is resolved or unresolved. In this case, there are 7 domains, 4 of which are unresolved.

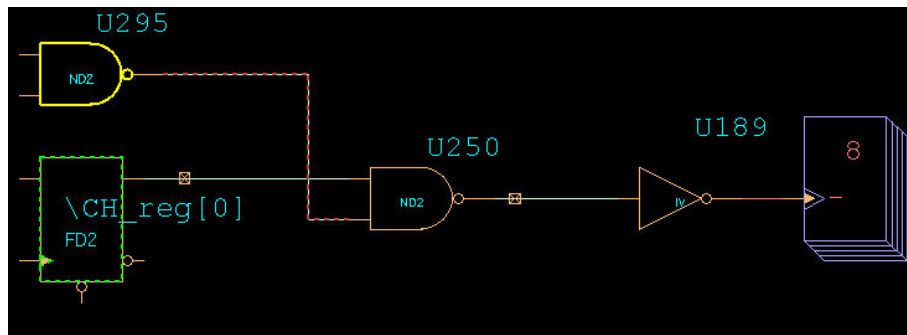
View Clock Domain Results in *nSchema*

1.  In the *Clock Domain* frame, select the *system.i_cpu.i_ALUB.S1 (f)* clock domain and click the **New Schematic** icon on the toolbar to open the *nSchema* frame. If an *nSchema* frame is already opened, you can alternatively drag the clock domain from the *Clock Domain* window and drop it in the *nSchema* frame.

The result is similar to Clock Tree Analysis in that like registers are grouped into a compressed register view.

2. In the *nSchema* frame for *system.i_cpu.i_ALUB.S1*, enable the **View -> Instance Name** toggle command.
3. Expand the inputs of ND2 U250 by double-clicking the pins. The schematic will look similar to the following:

Figure 40 Clock Domain in *nSchema*



The net associated with port *A* is the output of a flop. If the net associated with port *B* is assigned as a known constant signal, then *nAnalyzer* would be able to resolve this clock domain as *system.i_cpu.i_CCU.CH[0]*.

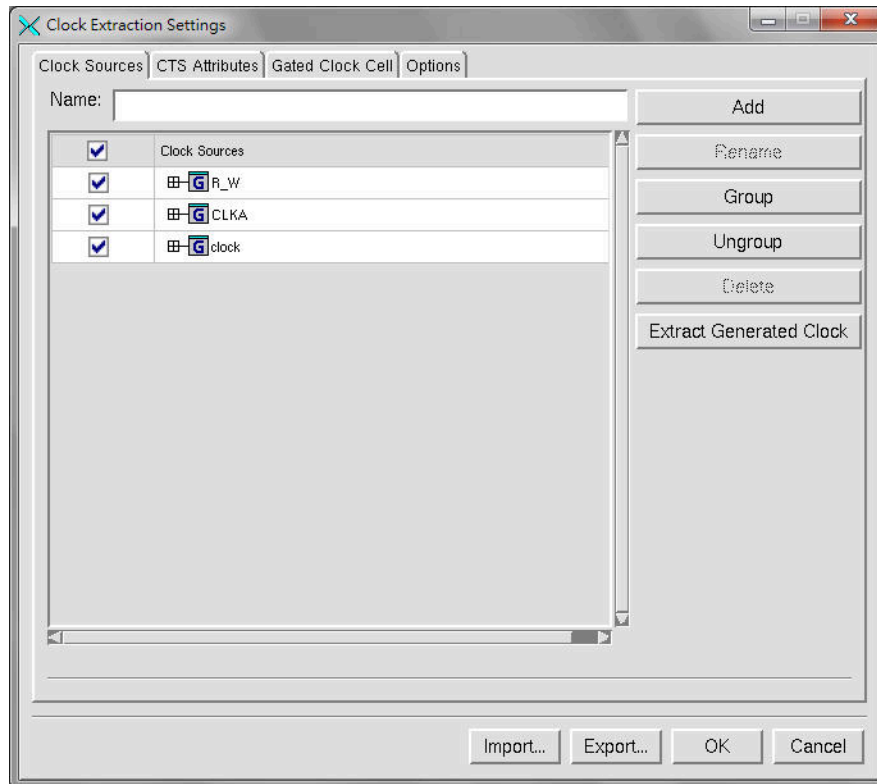
Further Constrain the Clock Domains

Let's modify some of the settings and reduce the list of domains. Since a Synopsys Design Constraint (SDC) file exists, let's start with that.

1. In the *nSchema* frame for *S1*, choose the **Trace -> Clock Analyzer -> Extract Clock Information** command.
2. In the *Clock Extraction Settings* form, click the **Import** button.
3. In the *Import* form, select **SDC** in the **Format** field.
4. Confirm the top design is *system* in the **Top Design** field, and select the *sdcsystem.sdc* file.
5. Click the **OK** button on the *Import* form.

Note that several known clock sources have been entered in the *Clock Extraction Settings* form, as shown below:

Figure 41 *Clock Extraction Settings Form After SDC Constraints are Loaded*



The known constants are available in the *Mode* form by clicking the **Mode** button in the **Options** tab of the *Clock Extraction Settings* form.

6. In the *Clock Extraction Settings* form, select the **Options** tab and then click the **Mode** button to open the *Mode* form.
7. Drag and drop the net (*n737*) associated with input *B* of *U250* in the *nSchema* frame for *S1* to the **Signal** tab on the *Mode* form.
8. Change the value to 1.

A value of 1 will allow the output to be traced through to the *A* input. A value of 0 will disable this gate completely.

9. Toggle the **Radix** option to **Binary**.

The form will be similar to the following:

Figure 42 Mode Form to Enable Known Constants

Mode

Editing Mode : default

Signal Cell Port

Signal	Value	Radix
system.reset	0	Binary
system.cpu1.ALUB.n737	1	Binary

Radix: Binary

Delete

Load from SDC...

Save New Mode

Mode List:

Mode Name

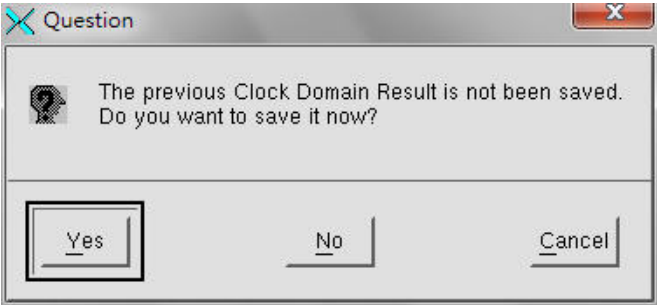
default

Load... Save... OK

10. In the *Mode* form, click the **OK** button.
11. In the **Clock Sources** tab of the *Clock Extraction Settings* form, click on the + symbol to the left of CLKA to see the clock sub-domains.
12. In the *Clock Extraction Settings* form, click the **Export** button.
An *Export* form opens.
13. Make sure the **Novas** option is selected in the **Format** field and enable the **All** option.
14. In the **File** field, specify *clocksrc.cs* and click the **OK** button.
15. Click the **OK** button in the *Clock Extraction Settings* form.

A *Question* dialog window opens, as shown below:

Figure 43 Question Dialog Window



16. Click the **No** button on the *Question* dialog window. The *Clock Domain* frame displays the results, as shown below:

Figure 44 New Extracted Clock Domain Results

Clock Source Tree	Number (Total:1)	Number of Flip-Flop (Total:105)	Resolve Type
clock		Total: 105	
system.clock	1		
system.clock (f)	1.f	46	Resolved (User Specified)
system.clock (r)	1.r	19	Resolved (User Specified)
system.i_cpu_i_ccu.C1	1.1		
system.i_cpu_i_ccu.C1 (r)	1.1.r	8	Resolved (Register Output)
system.i_cpu_i_ccu.C19	1.2		
system.i_cpu_i_ccu.C19 (f)	1.2.f	8	Resolved (Register Output)
system.i_cpu_i_ccu.C5	1.3		
system.i_cpu_i_ccu.C5 (f)	1.3.f	8	Resolved (Register Output)
system.i_cpu_i_ccu.C6	1.4		
system.i_cpu_i_ccu.C6 (f)	1.4.f	8	Resolved (User Specified)
system.i_cpu_i_ccu.CH[0]	1.5		
system.i_cpu_i_ccu.CH[0] (r)	1.5.r	8	Resolved (Register Output)

There are 7 clock domains, and they are all resolved.

17. Invoke the **File -> Exit** command on the *Clock Domain* frame.
18. The results can be displayed again by choosing the **Tools -> Clock Analyzer -> List Clock Domains** command in the main window or the **Trace -> Clock Analyzer -> List Clock Domains** command in any *nSchema* frame.

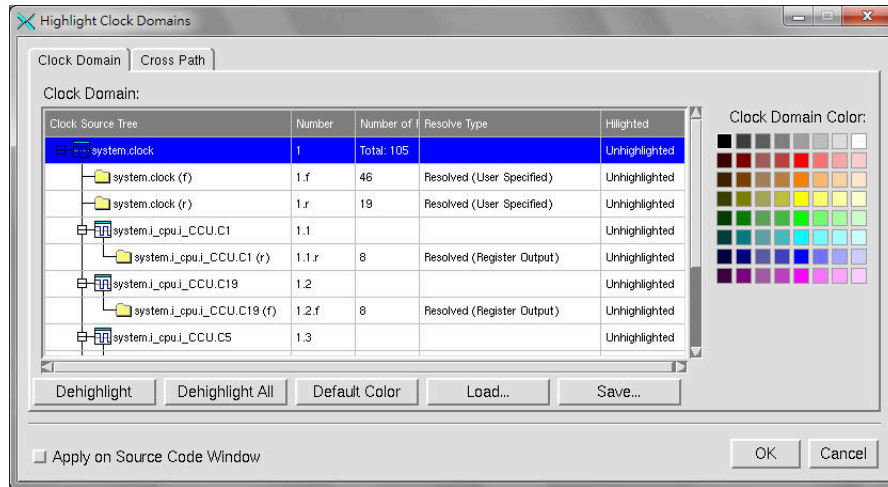
Highlight Clock Domains

After the clock domains are extracted, the different domains can be easily identified in the *nSchema* frame by changing the highlight color.

1. In the main window, choose the **Tools -> Clock Analyzer -> Highlight Clock Domains** command.

A *Highlight Clock Domains* form displays, as shown below:

Figure 45 Highlight Clock Domain Form



2. Change the color for some or all of the domains.
3. For example, change the color for *system.i_cpu.i_CCU.C5* to yellow.
4. Change the color for *system.i_cpu.i_CCU.C6* to turquoise.
5. Turn on the **Apply on Source Code Window** option.
6. Click the **OK** button on the *Highlight Clock Domains* form.

Find the Clock Source

In addition to extracting the clock domains on the entire design, the clock source can be found for an individual FF.

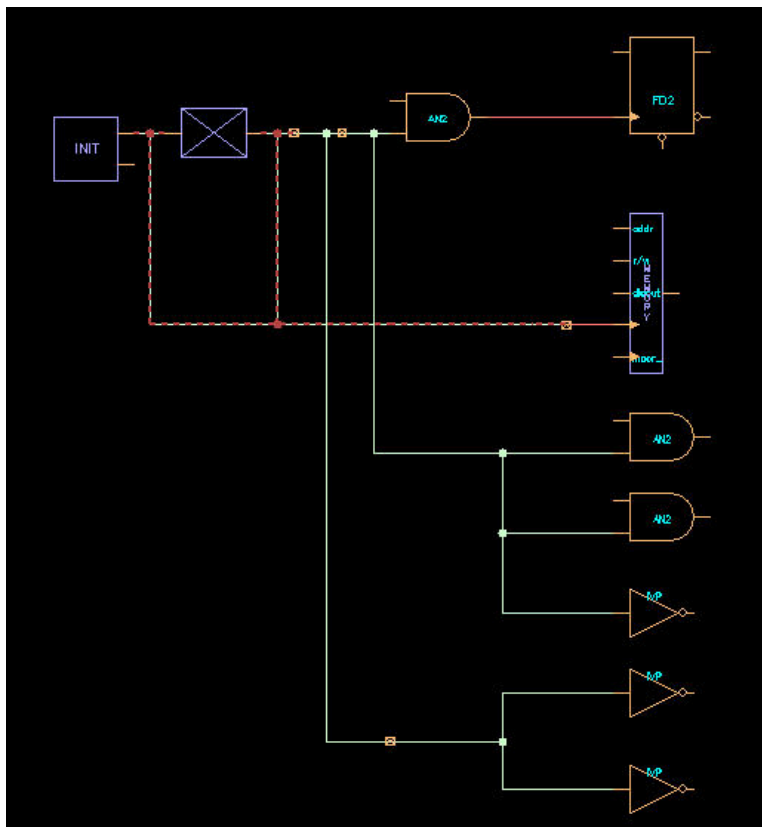
1. In the *nSchema* frame, locate the FF instance *carry_flag_reg* in the *i_ALUB* block.
 - a. In the **Inst._Tree** tab of the design browser frame, expand *i_cpu* and double-click on *i_ALUB* to display the source code.
 - b. Click the **New Schematic** toolbar icon to open the *nSchema* frame.
 - c. In the *nSchema* frame, enable the **Schematic -> Auto Fit Found Object(s)** toggle command.
 - d. Choose the **Schematic -> Find in Current Scope** command to open the *Find* form.
 - e. In the *Find* form, turn on the **Instance** option.

- f. Type *carry** in the **Find** text box and press enter on the keyboard to perform the string search.
 - g. Click *carry_flag_reg* and the register will be highlighted and selected in the *nSchema* frame.
2. Choose the **Trace -> Clock Analyzer -> Find Clock Source** command to find the clock source for the selected register.

This command will inherit the settings for clock domain extraction although they are not imported into the window. If the clock tree is not extracted first, you need to specify known clock signals.

3. In the *Find Clock Source* form, click the **Find** button. A new *nSchema* frame opens showing the path to the clock source.

Figure 46 Find Clock Source Results in *nSchema*



4. Close the *Find clock source result nSchema* frame.

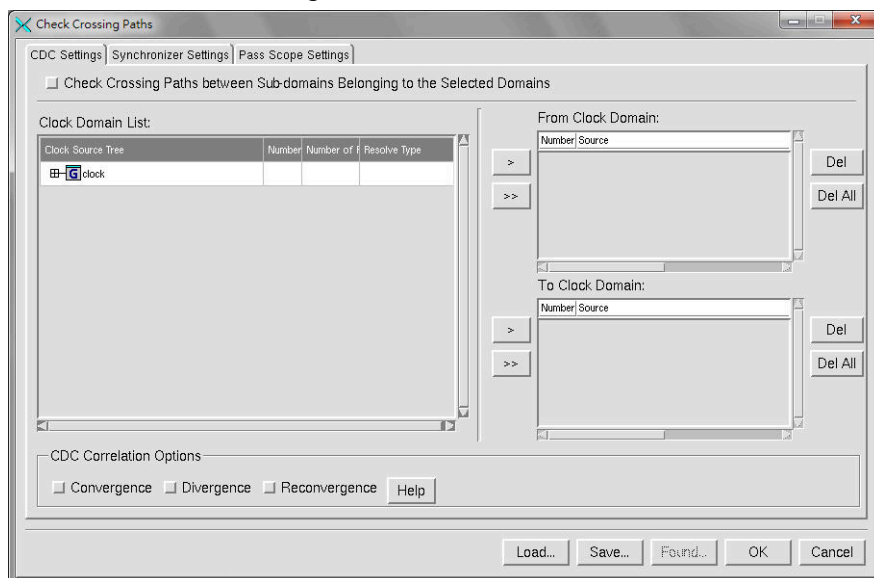
Analyze Clock Domains for Crossing Paths

Check for Crossing Paths

In the *nSchema* frame, after you **Extract Clock Domain**, you can further analyze paths crossing clock domains.

1. Continue from the previous example.
2. In any *nSchema* frame, choose the **Trace -> Clock Analyzer -> Check Crossing Paths** command. The *Check Crossing Paths* form displays, as shown below:

Figure 47 Check Crossing Paths Form



If you have a known synchronizer, you can specify it in this form on the **Synchronizer Setting** tab. Refer to [Appendix A: Synchronizer Template](#) for more details on the provided templates.

3. Expand all clocks under *clock* by clicking the + symbol and then expand *system.clock*.
4. Add *system.i_cpu.i_CCU.C6* (domain number 1.4) to the **From Clock Domain** field by selecting and clicking the > button.
5. Add *all clocks* by clicking the >> button to the **To Clock Domain** field. Only the top clock sources, in this case *clock*, get added to the list.

Note:

The sub-domains are also added but they are not displayed in the list. By default, crossing path checking only checks tree to tree. You can enable

the **Check Crossing Paths between Sub-domains Belonging to the Selected Domains** option to check between clock sub-domains as well.

- Click the **OK** button on the *Check Crossing Paths* form.

The results are shown in the *Check Crossing Paths* frame in the same location as the *Message* frame, similar to the following figure:

Figure 48 Crossing Paths Results

Crossing Paths:					
On	Path	From Clock Source	To Clock Source	With Synchronizer	Without Synchronizer
☒	Domain1.4f->Domain1.1r	system.i_cpu.i_CCU.C6(Falling)	system.i_cpu.i_CCU.C1(Rising)	0	64
☐	Domain1.4f->Domain1.3f	system.i_cpu.i_CCU.C6(Falling)	system.i_cpu.i_CCU.C5(Falling)	0	36
☐	Domain1.4f->Domain1.5r	system.i_cpu.i_CCU.C6(Falling)	system.i_cpu.i_CCU.CH[0](Rising)	0	36
☐	Domain1.4f->Domain1.1r	system.i_cpu.i_CCU.C6(Falling)	system.clock(Rising)	0	96

Register Pair Details:			
On	From Register	To Register	Synchronizer
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[0]	
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[1]	
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[2]	
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[3]	
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[4]	
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[5]	
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[6]	
☐	system.i_cpu.i_PCU.ATR_reg[0]	system.i_cpu.i_PCU.TDB_reg[7]	
☐	system.i_cpu.i_PCU.ATR_reg[1]	system.i_cpu.i_PCU.TDB_reg[0]	
☐	system.i_cpu.i_PCU.ATR_reg[1]	system.i_cpu.i_PCU.TDB_reg[1]	
☐	system.i_cpu.i_PCU.ATR_reg[1]	system.i_cpu.i_PCU.TDB_reg[2]	
☐	system.i_cpu.i_PCU.ATR_reg[1]	system.i_cpu.i_PCU.TDB_reg[3]	
☐	system.i_cpu.i_PCU.ATR_reg[1]	system.i_cpu.i_PCU.TDB_reg[4]	
☐	system.i_cpu.i_PCU.ATR_reg[1]	system.i_cpu.i_PCU.TDB_reg[5]	
☐	system.i_cpu.i_PCU.ATR_reg[1]	system.i_cpu.i_PCU.TDB_reg[6]	

The crossing paths are summarized in the top and the associated register pair details are on the bottom. The register pairs include paths from outputs of FFs/Latches in “From” domains to inputs of “To” domains with combinational logic in-between.

View Crossing Paths

Since multiple domains were selected, multiple paths with register pairs are listed.

- In the *Crossing Paths* form, choose the **Domain 1.4.f -> Domain 1.3.f** crossing path.
- Choose the **Tools -> Filter** command to open the *Filter* form.
- In the *Filter* form, select the **Register Pair Details** tab.
- Click the **Add** button to open the *Filter Rule Settings* form.
- Select *system.i_cpu.i_PCU.IDR_reg[7]* in the **To Register** column of the **Register Pair Details** table in the *Crossing Paths* form and use the middle mouse button to drag and drop it to the **To Register** field of the *Filter Rule Settings* form.

6. In the *Filter Rule Settings* form, toggle the **To Register Condition** to $\neq$. This will filter out everything except the specified register. The form will be similar to the following:

Figure 49 Filter Rule Setting Form

The screenshot shows the 'Filter Rule Settings' dialog box with the 'Programatic Conditions' tab selected. The 'False Path' tab is also visible. The 'To Register' condition is set to '!='. The 'To Register Data Signal' is set to 'n_i_cpu.i_PCU\TR_reg[0]'. The 'Filter Rule Connector' is set to 'AND'. A color palette is visible at the bottom left.

Field	Condition	Value
From Register:	null	
To Register:	!=	n_i_cpu.i_PCU\TR_reg[0]
From Register Data Signal:	null	
To Register Data Signal:	null	
Level:	null	
Synchronizer:	null	
Fan-in Number (Fan-in View Only):	null	
Fan-out Number (Fan-out View Only):	null	

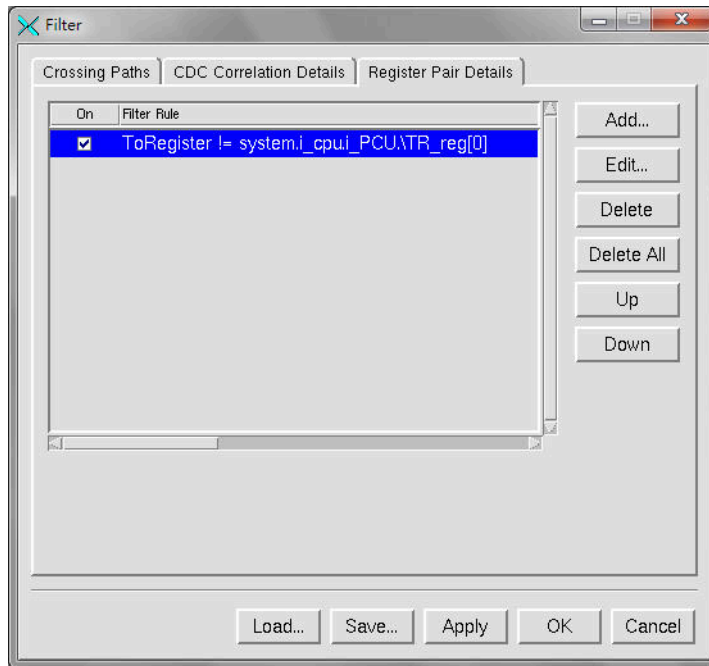
Filter Rule Connector: AND

Hidden

OK Cancel

- Click **OK** in the *Filter Rule Settings* form. The rule will be added to the *Filter* form, similar to the following:

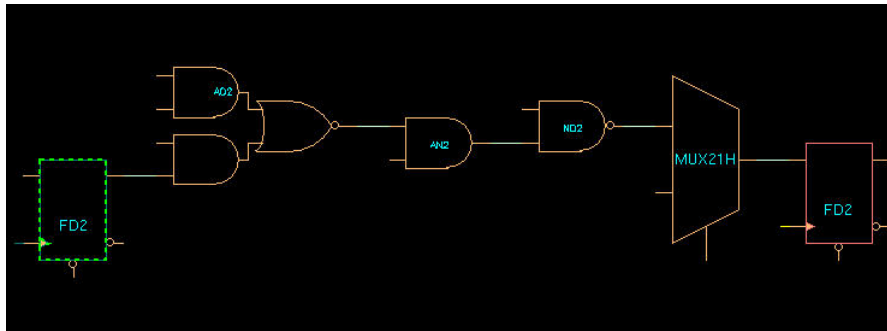
Figure 50 *Filter Form*



- Click the **OK** button in the *Filter* form. The list in the **Register Pair Details** section will be updated to match the filter.
- In the **Register Pair Details** section of the *Crossing Paths* form, select the 'From' .. \TR_reg[7] 'To' .. \IDR_reg[7] register pair (last option in the filtered list).

- Click the **New Schematic** icon on the toolbar to show in an *nSchema* frame. The *nSchema* frame will open similar to the following figure:

Figure 51 Crossing Path Results in *nSchema*



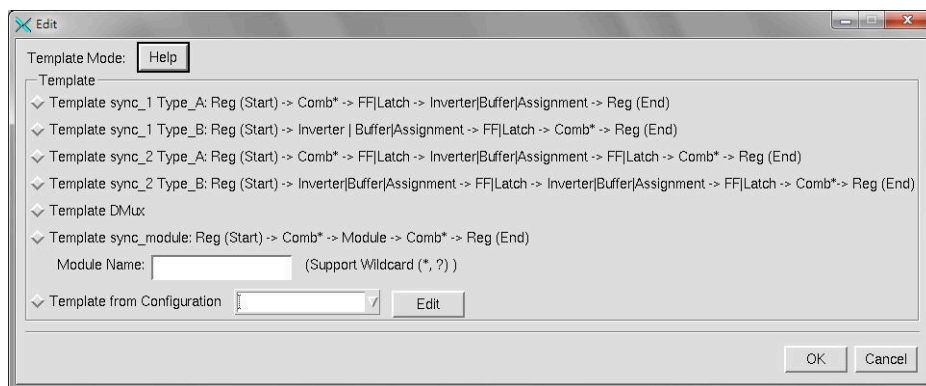
Note the two registers and the combinational logic in between. Multiple register pairs may be selected and shown simultaneously. The extracted clocks and checked crossing paths may be saved to a .cdb file for future reference.

Create a User-Defined Synchronizer

If you can't find the correct synchronizer from the list of pre-defined templates, you can define your own template.

- Continue from the previous example.
- In the main window, choose the **Tools -> Clock Analyzer -> Check Crossing Paths** command to open the *Check Crossing Paths* form.
- In the *Check Crossing Paths* form, click the **Synchronizer Settings** tab.
- Click the **Edit** button. The *Edit* form opens, similar to the following:

Figure 52 Edit Form



5. In the *Edit* form, select the **Template from Configuration** option and click the related **Edit** button.

The *Synchronizer Customization* form opens, similar to the following:

Figure 53 Synchronizer Customization Form

6. In the *Synchronizer Customization* form, click **New** to add a new template. The name defaults to NewXXX.
7. In the **Name** field, delete the default name and then enter a name for your template (e.g. mytemplate) and press enter on the keyboard.
8. In the **Synchronizer Configuration** section, select the **Any of Cells** option.
9. In the **Cell Type** field, left-click on the menu to see all the options and select **Comb**. Then click the <-button to move the selection to the **Cell Type** field.

Figure 54 Cell Type - Comb Selection

10. Click the **Append** button to add it to the path field.
11. In the **Synchronizer Configuration** section, select the **Synchronizer Flip-flop** option.
12. In the **Cell Type** field, left-click on the menu to see all the options, select **Flip-Flop** and click the **<**-button to move the selection to the **Cell Type** field.
13. Click the **Append** button to add it to the path field.
14. In the **Synchronizer Configuration** section, select the **Sequence of Cells** option.
15. In the **Cell Type** field, left-click on the menu to see all the options, select **Buffer** and click the **<**-button to move the selection to the **Cell Type** field.
16. Left-click again on the menu to see all the options, select **Assignment** and click the **<**-button to move the selection to the **Cell Type** field.
17. Click the **Append** button to add it to the path field. The path field section will be similar to the following:

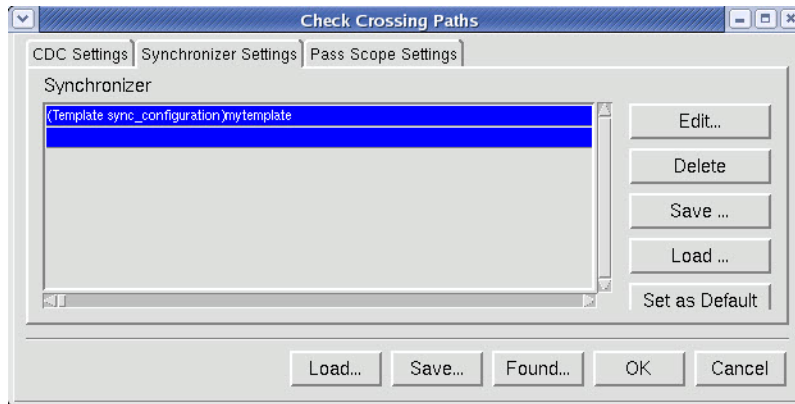
Figure 55 User-Defined Synchronizer

From	Path	To
Start FF	Comb	
	Flip-Flop	
	Buffer:Assignment	End FF

18. Click the **Commit** button to associate the configured template to the specified synchronizer name.
19. Click the **OK** button to apply the synchronizer for checking.

20. Click the **OK** button in the *Edit* form and the synchronizer template is added to the **Synchronizer** field. The form will be similar to the following:

Figure 56 Synchronizer List



Now you can continue checking paths.

4

Switching Analysis

This section describes the following topics:

- [Overview](#)
- [Tutorial](#)

Overview

As more applications require low power consumption and design schedules are compressed, waiting until the end of the design cycle for full power analysis is not always practical. With switching analysis, which correlates to power consumption, you can investigate signal activity early in the design cycle.

The two main requirements for switching analysis are a design (gate or RTL) with a corresponding FSDB file. After loading this information in the Verdi platform, then *nAnalyzer* module provides the following functionality:

- Determine which nets have the most transitions over a specified time range (switching activity).
- Identify which time has the most transitions (peak activity).
- Save the results for future review.

Tutorial

The following section is included:

- [Analyze Switching Activity on RTL Design](#)

Before you begin the tutorial, follow the instructions in the [Before You Begin](#) section of the *Introduction* chapter.

Analyze Switching Activity on RTL Design

This section includes the following topics:

- [Create Switching Activity Report](#)
- [Review the Switching Results](#)
- [Create Peak Activity Report](#)

Create Switching Activity Report

1. Change to the Verilog cpu directory:

```
% cd <working_dir>/demo/verilog/cpu
```

2. Load the design using the following command:

```
% verdi -play demo_verdi.cmd
```

3. In the **Inst._Tree** tab of the design browser frame, double-click *i_CPU*.
4. From the main window, choose the **Tools -> Switching Analysis -> New Query** command to open the *Switching Analysis* form.

The form will be similar to the following example:

Figure 57 Switching Analysis Form

Switching Analysis (on vgss3)

Working Scope:

Time Period:

From x 10ps To x 10ps

☐ Include Instances under the Hierarchy

☐ Include Clock Signals

☐ Perform Pattern Checking for Clock Analysis

☐ Report Top Record(s)

☐ Time Grouping for Peak Activity Report:

Time Window Size: x 10ps With Sliding Delta: x 10ps

☐ Output to File:

Save as XML File

Save as CSV File

Report Type: ☒ Switching Activity Report

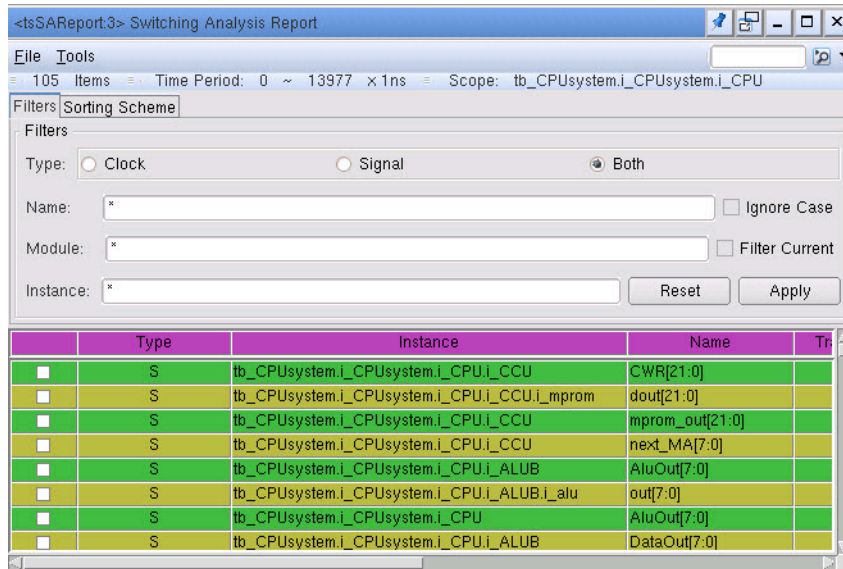
☐ Peak Activity Report

☐ Scope Based Peak Activity Report

5. In the *Switching Analysis* form, click the **Change** button next to the **Working Scope** field.
6. In the *Behavior Analysis* form, select the **Working Scope Mode** option and then click the **Browse** button.
7. In the *Instance Manager* form, expand *i_CPUsystem* and then select *i_CPU*.
8. Click the **OK** button and then click the **OK** button in the *Behavior Analysis* form.
9. In the *Switching Analysis* form, turn on the **Include Instances under the Hierarchy** option.
10. In the **Report Type** section, select the **Switching Activity Report** option. This will summarize the peak switching activity by signal.
11. Click the **OK** button.

Switching analysis is executed and the results are summarized on a signal basis in the *Switching Analysis Report* frame, similar to the following example.

Figure 58 Switching Analysis Report Frame



Review the Switching Results

1. In the *Switching Analysis Report* frame, select the **Filters** tab.
2. Enter **alu** in the **Module** text field, and click the **Apply** button.
This will list only the signals associated with the *i_alu* module.
3. Select the **Sorting Scheme** tab.
4. In the **Sorting Criteria** section, click the **Ascend** radio button on the first line and change the **Sort By** option to **Name**.

This will sort the filtered results by signal name, similar to the following example:

Figure 59 Switching Results - Filtered and Sorted

The screenshot shows the 'Switching Analysis Report' window. The 'Sorting Scheme' tab is active, displaying the following sorting criteria:

- Sort by: Name (Ascend)
- Then by: Module (Descend)
- Then by: Module (Descend)
- Then by: Module (Descend)

Below the sorting criteria is a table with the following data:

Type	Instance	Name	TransitionCount
S	tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu	cin	63
S	tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu	select[2:0]	19
S	tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu	out[7:0]	465
S	tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu	carry	27
S	tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu	a[7:0]	346
S	tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu	b[7:0]	80
S	tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu	zero	20

There are numerous ways the results can be manipulated to focus on areas of interest.

- Click-left on the **Transition Count** column to sort the results by the number of transitions. You'll see *out[7:0]* has the most transitions at 465.

Create Peak Activity Report

Let's take a look at the peak activity by time focusing on the *i_alu*.

- In the *Switching Analysis Report* form, click the **New Query** button.
- In the *Switching Analysis* form **Report Type** section, enable the **Peak Activity Report** option. This will summarize the peak switching activity by signal.

Assume you are only interested in activity after reset which de-asserts at time 215ns.

- Enter 215 in the **From** text field under **Time Period**.
- Click the **OK** button.

A new *Switching Analysis Report* frame opens as a new tab in the same location as the original *Switching Analysis Report* frame, displaying the peak activity by time and max transition count, similar to the following example.

Figure 60 Switching Analysis - Peak Results

	Time	TransitionCount
■	5976	93
■	2776	91
■	2876	86
■	3276	79
■	5526	72
■	2926	72
■	4476	71
■	6076	68
■	5776	68
■	276	67
■	13976	67
■	7776	66
■	3076	65
■	1776	64
■	4301	64
■	5276	63
■	13176	62
■	6676	62

5. In the main window, choose the **File -> Exit** command to exit the Verdi platform.

5

Appendix A: Synchronizer Template

This section describes the following topics:

- [Overview](#)
- [Elements in Synchronizer Template](#)
- [Template Types](#)
- [Default Template](#)

Overview

This appendix introduces the synchronizer template, which is used for the following purposes:

- To verify mixed clock domain paths are synchronized correctly.
- To find out which paths satisfy some user-specified rules.
- To report violations when a path violates some user-specified rules.

In real designs, there are many kinds of synchronizers which can be used to synchronize the data propagation between different clock domains. Some synchronizers are simple and some are very complex. The template is a set of constraints. It will not use dynamic values such as stimuli of design and simulation results.

Elements in Synchronizer Template

A synchronizer template may contain the following major elements.

StartPoint

The start point is the output of sequential logic such as a flop or latch or primary input.

EndPoint

The end point is the input of sequential logic such as a flop or latch or primary output.

CrossPath

The cross path connects a StartPoint and an EndPoint if the StartPoint and EndPoint belong to different clock domains. Generally, it can be achieved by checking mixed clock domain paths in the design.

FromDomain

A From Domain can be any clock domain or level-sensitive (latch) clock domain in the design. It must be specified by domain name.

ToDomain

A To Domain can be any clock domain or level-sensitive (latch) clock domain in the design. It must be specified by domain name.

Template

The template is a set of user-specified constraints for proving. The proving action will be applied to the CrossPath.

Template Types

For single bit data propagating between asynchronous clock domains, the synchronizer is used to reduce the probability for the ToDomain to sample a meta-stability value. In such cases, the synchronizer could be simple.

For bus data propagating between asynchronous clock domains, the synchronizer is not only used to resolve the meta-stability issue, but also the synchronization between different bits if more than one bit of the bus changes value at the same clock edge. In such cases, the communication protocol should be introduced into the synchronizer; so the synchronizer will become very complex.

If the data is propagating from a fast clock domain to slower clock domain, the synchronizer may be used to avoid missing data in the ToDomain. In such cases, the communication protocol should be introduced into the synchronizer and it will become very complex.

Here we can only prove the simple synchronizer using the template. The constraints are used to verify the simple synchronizer. We extended the use of constraints by design hierarchy. If the synchronizer (simple or complex) is described in a module (cell), we can check whether the crossing path passes through the synchronizer module (cell) or not.

There are two types of templates. One is a set of constraints specifying the detailed cells in the crossing path, which is called a flatten synchronizer template. The other is the synchronizer module.

Synchronizer Module Template

It is a good coding practice to enclose a protocol-based synchronizer into an isolated module considering:

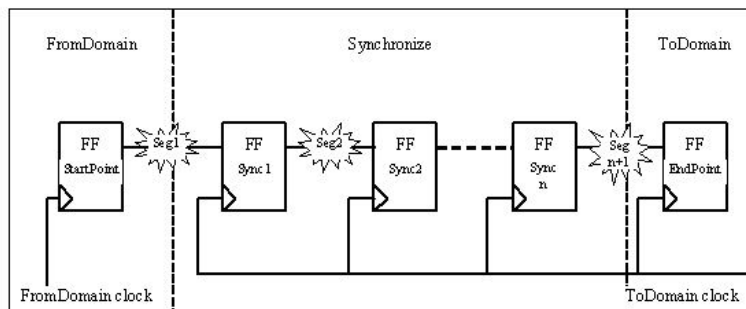
- Code reuse.
- Design verification and debug.

If the protocol to synchronize data between different clock domains is written in an isolated module, you can check whether the crossing path passes through the module or not. The only thing to be customized is the synchronizer module names.

Flatten Synchronizer Template

The main purpose for this template is to reduce the meta-stability issue. So the primary concern is the flip-flop in the path. Together with the StartPoint and EndPoint, the path is divided into several segments as shown in the following figure.

Figure 61 Synchronizer Templat



In the above figure, FF indicates a flip-flop or latch and Seg1, Seg2, Seg n+1 are the segments between two FF and they can only be combinational logic cells.

The constraints specify the following:

- The number of synchronizer FF (flip-flop or latch).
- The combinational cell types (or cell names) in the segments.

In addition to customizing the above constraints, the following constraints should also be satisfied:

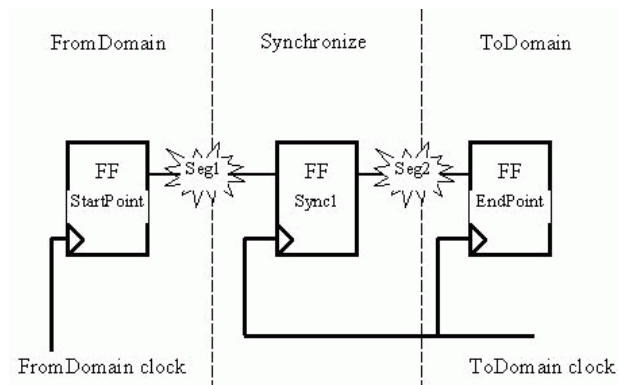
- All the clock ports of the FF in the synchronizer section should be connected to the clock signal of the ToDomain.
- The fan out of all FF in the synchronizer section should not exceed one.

Default Template

The following default templates are provided.

Template sync_1 Type A & B

Figure 62 Template sync_1



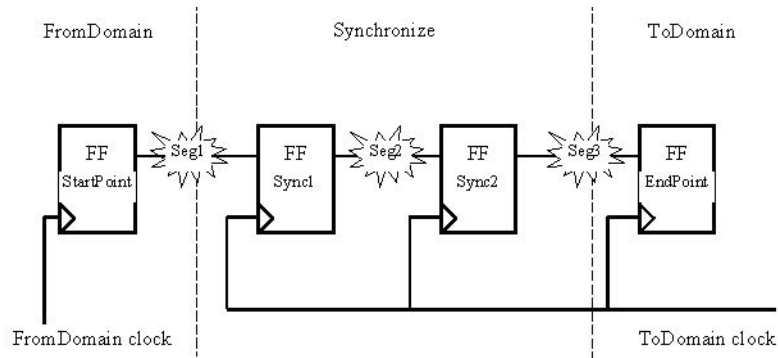
In this template, the following constraints should be followed:

- At least one synchronizer FF (flip-flop or latch) between the asynchronous clock domains.
- The clock port of the synchronizer FF Sync1 should be connected to the clock signal of the ToDomain.
- Type A: In Seg1, there can be zero or any number of combinational logic cells.
- Type B: In Seg1, only a buffer/inverter/assignment is allowed.

- Type A: In Seg2, only a buffer/inverter/assignment is allowed.
- Type B: In Seg2, there can be zero or any number of combinational logic cells.

Template sync_2 Type A & B

Figure 63 Template sync_2

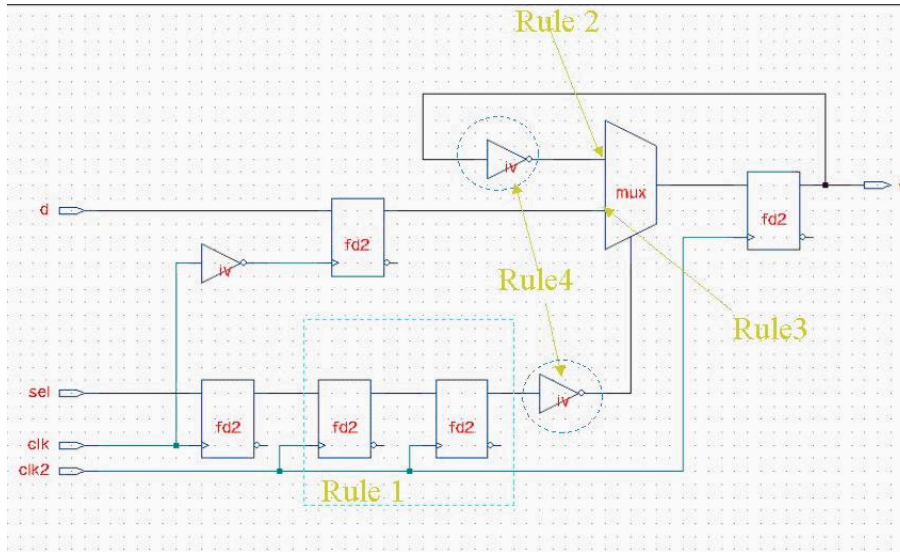


In this template, the following constraints should be followed:

- At least two synchronizer FF's (flip-flop or latch) between the asynchronous clock domains.
- The clock port of the synchronizer FF Sync1 should be connected to the clock signal of the ToDomain.
- The fan out of Sync1 in the synchronizer section should not exceed one.
- Type A: In Seg1, there can be zero or any number of combinational logic cells.
- Type B: In Seg1, only a buffer/inverter/assignment is allowed.
- In Seg2, only a buffer/inverter/assignment is allowed.
- In Seg3, there can be zero or any number of combinational logic cells.

Mux (D-Mux) Synchronizer

Figure 64 D-Mux Synchronizer

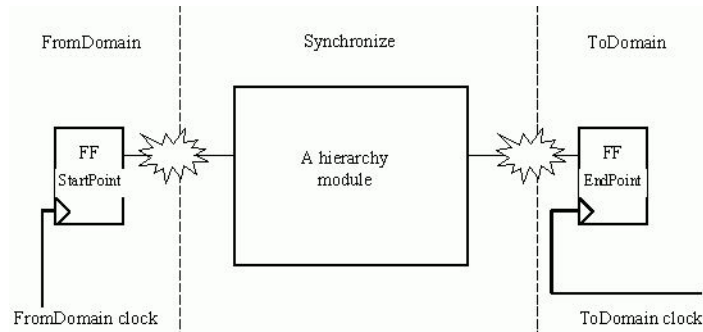


In this template, the following constraints should be followed:

- The selected input of MUX comes from the To domain, and is synchronized by two flip-flops (control path synchronizer). See Rule 1 as an example.
- One of the MUX inputs comes from the To domain via the holding loop. See Rule 2 as an example.
- The MUX inputs are From domain, To domain or user-specified static signals. See Rule 3 as an example.
- The logic between the MUX synchronizer and the To flop is driven by the To domain or static signals. See Rule 4 as an example.

Template sync_module

Figure 65 Template sync_module



In this template, the module name should be configured before it is used for synchronizer proving. The module name may be changed according to the design. The only constraint is that the crossing path should pass through the specified synchronizer module.

Note:

Limitation: In the synchronizer module, there must be synchronizer flip-flop or latch on the crossing path. Otherwise, the crossing path cannot be proved to be synchronized even though it is passing through the synchronizer module.

6

Appendix B: CDC Correlation

This section describes the following topics:

- [Convergence](#)
- [Divergence](#)
- [Reconvergence](#)

Convergence

Different start registers are converged to a point and then stop at the same end register.

Figure 66 Convergence Before Synchronizer

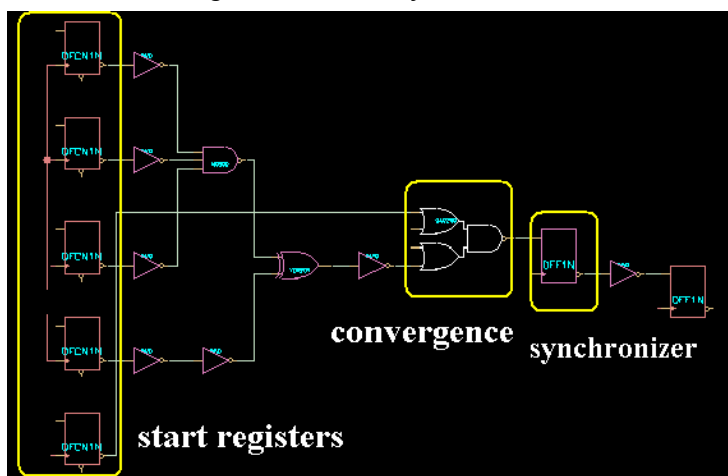
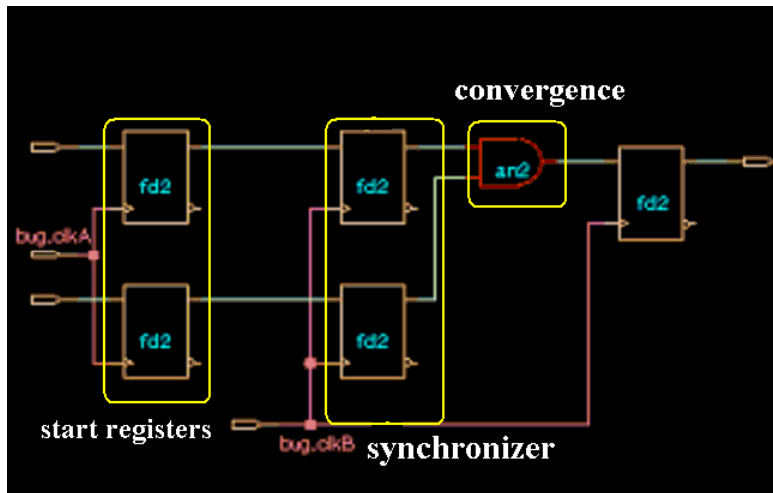


Figure 67 Convergence After Synchronizer



Divergence

One start register diverges to different end registers.

Figure 68 Divergence Before Synchronizer

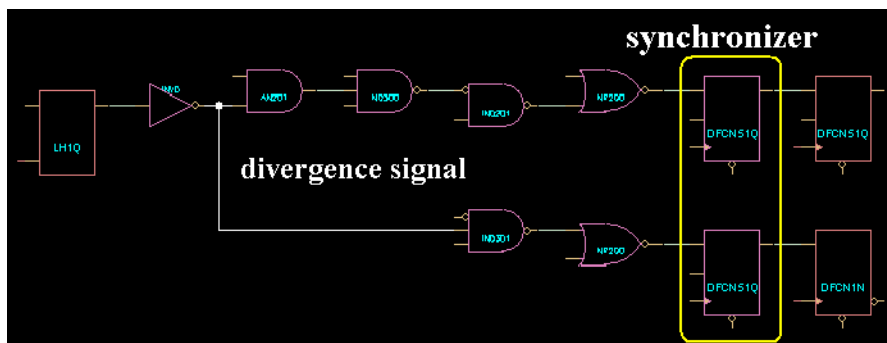
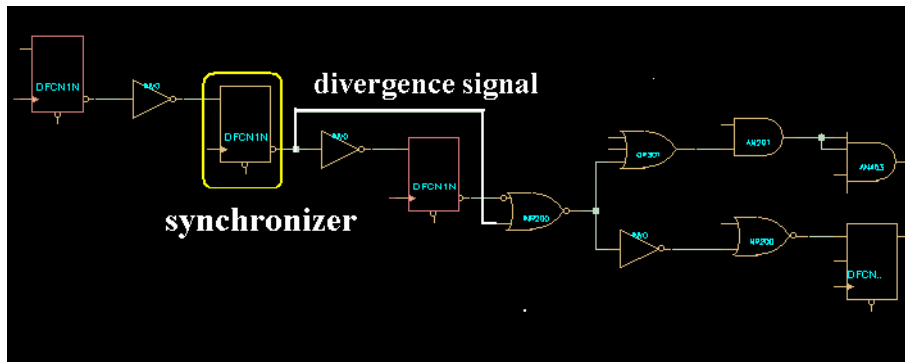


Figure 70 Reconvergence



Verdi® nAnalyzer
W-2024.09

81

Figure 70 Reconvergence

